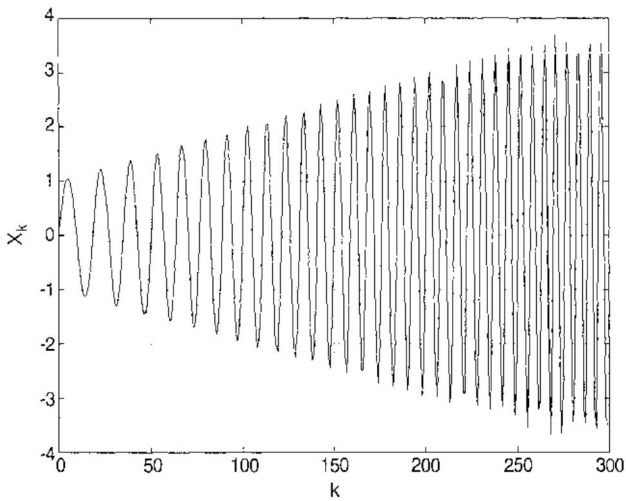

FIGURE 6.19
Plot of 300 samples of a nonstationary signal increasing in both frequency and amplitude.

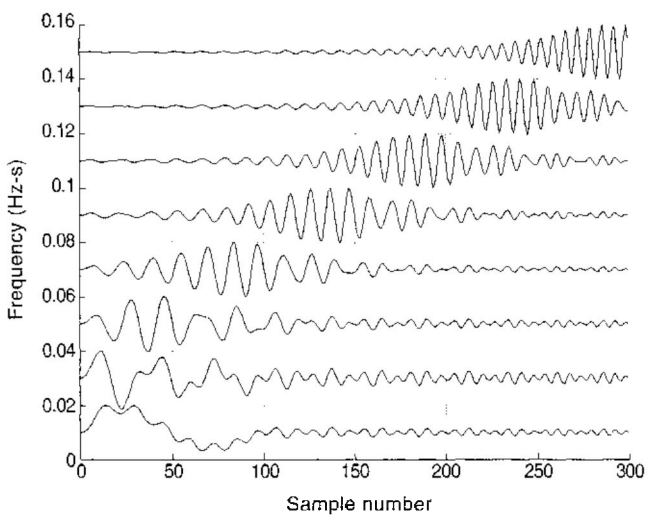

FIGURE 6.20
Spectrogram of the signal in Figure 6.19 produced using a comb filter with $N = 100$, $r = 0.99$, and resonances at $[1:2:16] * (2 * \pi/N)$.

We can see that the amplitude response is one at all frequencies:

$$|H(e^{j\omega T})| = \frac{|pe^{j\omega T} - 1||p'e^{j\omega T} - 1|}{|e^{j\omega T} - p||e^{j\omega T} - p'|} = \frac{|pe^{j\omega T} - 1||p'e^{j\omega T} - 1|}{|1 - pe^{-j\omega T}||1 - p'e^{-j\omega T}|}$$
$$= \frac{|pe^{j\omega T} - 1||p'e^{j\omega T} - 1|}{|(1 - p'e^{j\omega T})'||(1 - pe^{j\omega T})'|} = 1 \qquad (6.57)$$

The operations in (6.57) were first to multiply each denominator term by $|e^{-j\omega T}| = 1$, and then use the distributive property of the conjugate plus the property $|\pm\xi'| = |\xi|$ for any complex variable or function ξ. Thus, the allpass filter is a filter with unit gain, and phase shift depending on the value of p. Allpass filter sections, each with its own value of p, may be connected in cascade to produce different phase characteristics. For further analysis of allpass phase and group-delay characteristics, the reader is referred to more advanced DSP texts, such as Oppenheim and Schafer.[5]

6.9 Exercises

General Instructions: (1) Whenever you make a plot in any of the exercises, be sure to label both axes so we can tell exactly what the units mean. (2) Make continuous plots unless otherwise specified. "Discrete plot" means to plot discrete symbols, unconnected unless otherwise specified. (3) In plots, and especially in subplots, use the *axis* function for best use of the plotting area.

1. Make two subplots. On the left subplot, plot the s-plane poles of an analog lowpass Butterworth filter having $L = 12$ poles and cutoff frequency equal to 2.0 rad/s. On the right, plot the power gain from −200 to 0 dB vs rad/s from 0 to 10.
2. Do Exercise 1 using a Chebyshev filter. The stopband should begin at 5 rad/s. Plot zeros as well as poles on the s-plane.
3. Make two subplots. On the left, plot the power gains of four digital lowpass Butterworth filters with cutoff at 0.3 Hz-s and $L = 2$, 4, 8, and 16. Plot dB in the range [−120, 0] vs. Hz-s in the range [0, 0.5]. On the right, plot the same curves for the corresponding Chebyshev filters with stopband gain set to −100 dB.
4. Make two subplots. On the left, plot the poles and zeros of a digital Butterworth filter with $L = 10$ poles and passband from 80 to 100 KHz, assuming a time step of 0.005 ms. On the right, make the same plot for a Chebyshev filter, assuming stopband gain = −80 dB.

5. Complete the following:
   a. Make two subplots. On the left, plot the amplitude gain vs. frequency in KHz for a digital Butterworth filter with $L = 10$ poles and passband from 80 to 100 KHz, assuming a time step of 0.005 ms. On the right, make the same plot for a Chebyshev filter, assuming stopband gain = –80 dB.
   b. Explain the forms of the plots in terms of the pole and zero locations in Exercise 4.
   c. Comment on the differences between the two amplitude gain plots.
6. Make two subplots. On the left, plot the amplitude gain vs. frequency in MHz for a digital Butterworth filter with 16 poles and passband from 70 to 90 MHz, assuming a sampling frequency of 300 MHz. On the right, make the same plot for a Chebyshev filter, assuming stopband gain = –80 dB.
7. Repeat Exercise 6, plotting power gain in the range [–100, 0] dB instead of amplitude gain.
8. Make two subplots. Plot the phases of the two filters in Exercise 6 over just the passband from 70 to 90 MHz. In both plots, plot phase shift in the range [–10, 20] rad.
9. An analog filter has s-plane poles at –4 and $-2 \pm j4$, zeros at $\pm j$, and a maximum gain of 1. Make two subplots. On the left, plot the amplitude gain vs. frequency in the range [0, 50] rad/s. On the right, plot amplitude gain vs. frequency in the range [0, 0.5] Hz-s for the digital filter obtained via the bilinear transformation. Explain how the analog gain peak at 4.0 rad/s is mapped to the digital gain peak at 0.42 Hz-s.
10. Generate a signal vector x with 200 samples of $x(t)$, which consists of a unit sine wave at 5 KHz plus another sine wave with amplitude 10 at 15 KHz, assuming a time step of 0.025 ms. Design a Butterworth digital filter with cutoff at 10 KHz and power gain down at least 60 dB at 15 KHz.
    a. Find L, the necessary number of poles.
    b. Design the filter and plot power gain in the range [–100, 0] dB vs. frequency in KHz. Write your prediction of the effect of the filter on x.
    c. In a separate figure, make two subplots. In the upper subplot, plot x vs. time in ms. In the lower subplot, plot the filtered version of x.
11. Test a digital highpass Chebyshev filter by filtering a sinusoid with frequency equal to the cutoff frequency. Use $L = 8$, dB = –100, and $v_c = 0.05$ in your test, and filter 200 samples of a unit sine wave. Plot the input and output signals together.

a. Comment on whether or not the filter gain is correct at the cutoff frequency.

b. Why are the two signals out of phase?

c. How would you estimate the approximate duration of the filter's impulse response?

12. A signal $x(t)$ has unit sinusoidal components at 10, 20, 30, 40, and 50 Hz. A two-second segment of $x(t)$ is sampled at 400 samples/s. Make three subplots, each with frequency range [0, 80] Hz.

a. On the left, plot the amplitude spectrum, $|X|$.

b. In the middle, plot the amplitude gain of a digital Chebyshev bandstop filter designed to eliminate just the component at 40 Hz.

c. On the right, plot the amplitude spectrum of the filtered version of x, indicating the effect of the filter.

13. Make three subplots. On the upper left, plot the power gain in the range [–80, 0] dB of a highpass digital Butterworth filter with cutoff at 0.3 Hz-s. On the upper right, plot the poles and zeros of this filter. Construct a sample vector with 200 samples of the sum of two sine waves: $x = 10\sin(2\pi(.1)k) + \sin(2\pi(.35)k)$; $0 \le k \le 199$. Filter x to produce y. In the lower subplot, plot x and y together.

14. Construct a comb filter in the configuration of Figure 6.17. Use N = 100 zeros and pole/zero radius r = 0.99. Set the poles for resonance at frequencies $[1:2:16]/N$ Hz-s. Make a plot similar to Figure 6.18, with a pole-zero plot on the left and a set of power gain plots on the right. Plot power gain in the range [–40, 0] dB vs. frequency in the range [0, 0.5] Hz-s.

15. Generate 300 samples of the chirping signal in Figure 6.19 with the following expressions:

```
k=0:299; x=(1+3*k/300).*sin(2*pi*k.*(.05+(k/300)*.06));
```

a. Plot the signal and observe that it is the same as Figure 6.19.

b. In a separate plot, reproduce the spectrogram in Figure 6.20 using the comb filter in Exercise 14. Plot each waveform with an offset corresponding with the resonant frequency of the comb filter section.

16. Design a Butterworth lowpass digital filter with L = 10 poles and cutoff at 0.4 Hz-s. Plot the power gain in dB vs. frequency in Hz-s. Set the axis ranges to [–80, 0] dB and [0, 0.5] Hz-s, respectively. Then on the same plot, overlay the power gain of an FIR lowpass filter using the Kaiser window with $\beta = 8.0$ that matches the Butterworth gain as closely as possible. Make the FIR plot with 50 connected discrete points. Choose the FIR cutoff frequency (ν_c) and filter length (N) to produce the best match. Print your choices in the plot title.

References

1. Orfanidis, S.J., *Introduction to Digital Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 9.
2. Mitra, S.K. and Kaiser, J.F., Eds., *Handbook of Digital Signal Processing*, Wiley, New York, 1993.
3. Antoniou, A., *Digital Filters: Analysis and Design*, 2nd ed., McGraw-Hill, New York, 1993.
4. Jackson, L.B., *Digital Filters and Signal Processing*, Kluwer Academic Publishers, Norwell, MA, 1989.
5. Oppenheim, A.V. and Schafer, R.W., *Discrete-Time Signal Processing*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 1998, chap. 5.
6. Stearns, S.D. and Hush, D.R., *Digital Signal Analysis*, Prentice Hall, Englewood Cliffs, NJ, 1989, chap. 12.
7. Parks, T.W. and Burrus, C.S., *Digital Filter Design*, Wiley, New York, 1987.
8. Roberts, R.A. and Mullis, C.T., *Digital Signal Processing*, Addison-Wesley, Reading, MA, 1987.
9. Elliott, D.F., Ed., *Handbook of Digital Signal Processing*, Academic Press, New York, 1987.
10. Rabiner, L.R. and Rader, C.M., Eds., *Digital Signal Processing*, IEEE Press, New York, 1972.
11. Kaiser, J.F., Digital filters, in *System Analysis by Digital Computer*, Kuo, F.F. and Kaiser, J.F., Eds., John Wiley & Sons, Inc., New York, 1966, chap. 7.
12. Kaiser, J.F., Some practical considerations in the realization of linear digital filters, *Proc. 3rd Ann. Conf. Circuits System Theory*, 621, 1965.
13. Storer, J.E., *Passive Network Synthesis*, McGraw-Hill, New York, 1957, chap. 30.
14. Butterworth, S., On the theory of filter amplifiers, *Wireless Engr.*, 1, 536, 1930.
15. Lutovac, M.D., Tosic, D.V., and Evans, B.L., *Filter Design for Signal Processing Using MATLAB and Mathematica*, Prentice Hall, Upper Saddle River, NJ, 2001.
16. Proakis, J. and Manolakis, D., *Digital Signal Processing: Principles, Algorithms, and Applications (3rd ed.)*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 8.

7

Random Signals and Spectral Estimation

7.1 Introduction

Random signals are signals with elements that cannot be predicted or derived exactly from other elements. In our previous examples of discrete signals, we used finite data vectors or analytic signals that are either periodic or transient; that is, we used signals that can be described completely in simple terms.

When a signal has random components and cannot be described analytically, we may be able to describe it in terms of its *statistical properties*, which we discuss in this chapter. There are two basic sources of the most commonly used statistical properties. One is the *amplitude distribution* of the random signal, and the other is the autocorrelation function or, equivalently, the *power density spectrum*. The amplitude distribution is something we have not yet considered. The autocorrelation function is an application of the correlation function in Chapter 3, Section 3.2. The power density spectrum is like the power spectra we have been using to analyze filters, but not quite the same thing, as we shall see.

Before we discuss these representations of the signal, we should consider the assumption of *stationarity* that is made or implied when statistical signal processing techniques are applied in DSP. A *stationary* signal is a signal with local statistics that are invariant over the entire duration of the signal. Thus, a periodic signal is a stationary signal, but a transient signal that occurs locally in a long time domain is not stationary. A random signal is stationary if average local estimates of the amplitude distribution and the autocorrelation function do not change over the duration of the signal. Figure 7.1 shows an example of a stationary signal with two nonstationary signals. The upper signal is nonstationary because its average power is steadily increasing. The middle signal is nonstationary because one of its components is steadily increasing in frequency, thus we could say that its power spectrum is changing with time. Only the lower signal is stationary, with unchanging distributions of amplitude and power.

One could argue that no process in the natural world is truly stationary. But, there are many situations in engineering where the *assumption* of stationarity

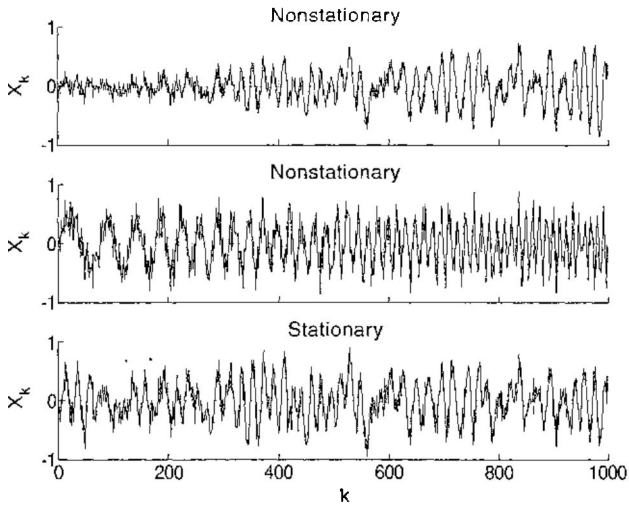

FIGURE 7.1
Vectors taken from nonstationary and stationary signals. The upper signal is nonstationary, because its average power is increasing steadily. The center signal is nonstationary, because its frequency is increasing steadily. The lower signal is stationary.

can be very useful. For example, if a long recording of the signal from a noise source is obtained, the estimated *power density spectrum*, which we describe in this chapter, is usually the best way we have to characterize the signal.

In this chapter, we begin by discussing the amplitude distributions of random signals and then go on to discuss power spectral estimates.

7.2 Amplitude Distributions

Suppose we have a long recording of a signal, such as a measurement of temperature, pressure, radiation from a particular source, pulse rate (there are many examples like these), and suppose the recorded signal is stationary and includes random components. If the signal is digitized and processed using DSP techniques, we may consider two kinds of *amplitude distributions*,[1–5] that is, functions that give the relative likelihood, or *frequency*, of different signal amplitudes. (This is a different use than we have had for "frequency," but also a common use of the word in statistical analysis.) To analyze the continuous signal before it is digitized, we use the *continuous amplitude distribution*. To analyze the signal after it has been digitized and can only have discrete values, we use the *discrete amplitude distribution*.

These two cases are shown in Figure 7.2. For the continuous signal, all values of $x(t)$ in the range [–7, 7] are theoretically possible. But, the digitized

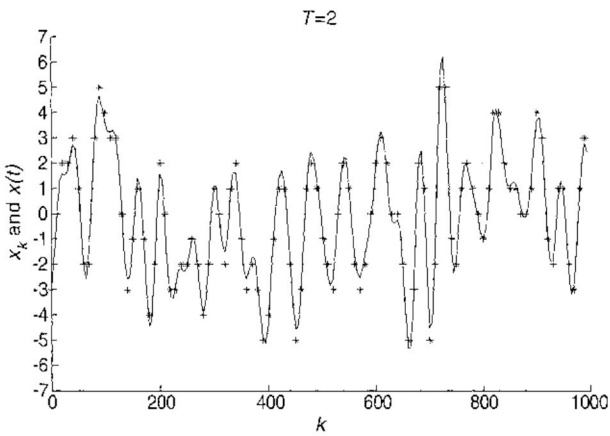

FIGURE 7.2
Stationary random waveform with discrete samples. Each sample, x_k, is the integer nearest to the corresponding waveform value, $x(kT)$.

signal can only have integer values in this range; that is, each x_k is the integer nearest $x(kT)$. The continuous and discrete amplitudes are distributed in accordance with the following functions:

$$\begin{aligned} &\text{Continuous:} \quad p(x); \quad \int_{-\infty}^{\infty} p(x)\,dx = 1 \\ &\text{Discrete:} \quad P_n = Pr\{x = n\}; \quad \sum_{n=-\infty}^{\infty} P_n = 1 \end{aligned} \tag{7.1}$$

Thus, $p(x)dx$ is the *probability* that $x(t)$ is in the range $[x, x + dx]$, and P_n is the probability that x takes the integer value n. The function $p(x)$ is called the *probability density* function of x. Thus, assuming the analog-to-digital conversion is unbiased, we may say that

$$P_n = \int_{n-1/2}^{n+1/2} p(x)dx \tag{7.2}$$

In signal processing applications, the discrete probability function is often estimated as the *frequency function*, f_n, which is simply the relative frequency of occurrence of integer n in a digitized sample vector; that is, for a digitized sample vector of length N,

$$P_n \approx f_n = \frac{\#\text{ samples equal to } n}{N} \tag{7.3}$$

Before examining important amplitude distributions, specific examples of $p(x)$ and P_n, a few fundamental properties may be defined in general terms. First, the *mean value* (*expected value, average value*) of any function of x, say $y(x)$, is defined as follows:

$$\begin{aligned} \text{Continuous:} \quad E[y] &= \int_{-\infty}^{\infty} y(x)p(x)dx \\ \text{Discrete:} \quad E[y] &= \sum_{n=-\infty}^{\infty} y(x_n)P_n \end{aligned} \tag{7.4}$$

Thus, $E[y]$ is essentially the sum of all values of $y(x)$, each value being weighted by $p(x)$. If the integral or sum in (7.4) does not converge, then $y(x)$ has no mean value. Two particular mean-value functions are so important that they are given special symbols. First is the mean value of x, given the symbol μ_x. Applying the continuous form of (7.4) with $y(x) = x$, the mean value of x is

$$\mu_x \equiv E[x] = \int_{-\infty}^{\infty} xp(x)dx \tag{7.5}$$

Thus, μ_x is essentially the sum of weighted values of x, that is, the "center of mass" coordinate of the distribution given by $p(x)$.

The second important expected value function is the *variance* of x, which is given the symbol σ_x^2. The variance is the most common measure of the variability of $x(t)$ about its mean value, μ_x. It is defined as the expected squared deviation of x from its mean value, i.e., σ_x^2 is the expected value of $y(x) = (x - \mu_x)^2$. Accordingly,

$$\begin{aligned} \sigma_x^2 &= E[(x-\mu_x)^2] = \int_{-\infty}^{\infty} (x-\mu_x)^2 p(x)dx \\ &= \int_{-\infty}^{\infty} x^2 p(x)dx - 2\mu_x \int_{-\infty}^{\infty} xp(x)dx + \mu_x^2 \int_{-\infty}^{\infty} p(x)dx \\ &= E[x^2] - 2\mu_x\mu_x + \mu_x^2 = E[x^2] - \mu_x^2 \end{aligned} \tag{7.6}$$

Thus, the variance turns out to be the difference between the mean squared value of x and the square of μ_x. The square root of, σ_x^2, σ_x, called the *standard deviation* of x, is the standard measure of the deviation of x from its mean value, the measure having the same units as x.

An important related problem arising often in signal analysis is that of determining the probability density of a function $y(x)$, given $p(x)$. This problem is solved in general[1] by partitioning x into segments over which $y(x)$ is monotonic and viewing the product $p(x)dx$ defined in (7.1) as the probability that $x(t)$ lies between x and $x + dx$, that is, viewing $p(x)dx$ as a vanishingly small increment of area above the sample space of x. In the simplest case,

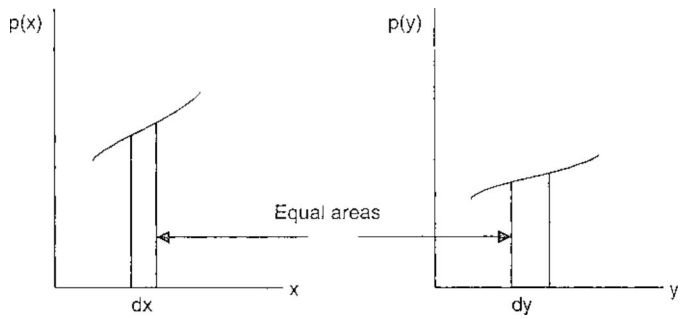

FIGURE 7.3
Equal probabilities shown as equal areas.

which is illustrated in Figure 7.3, $y(x)$ is a unique, one-to-one mapping between the x and y sample spaces, so that $p(x)$ and $p(y)$ are different functions, but $p(x)dx$ and $p(y)dy$ are the same probabilities. That is, assuming x and y are differentiable,

$$\text{Equal areas:} \quad p(y)|dy| = p(x)|dx|, \quad \text{or} \quad p(y) = \frac{p(x)}{|dy/dx|} \tag{7.7}$$

where $|dy/dx|$ is the absolute value of the derivative of $y(x)$. Exercises 1 and 2 provide illustrations of this relationship.

The case where y is a linear function of x, say $y = ax + b$, is another useful example. In this case $|dy/dx| = |a|$, and so $p(y) = p(x)/|a|$. Two useful corollaries follow from this result:

$$\begin{aligned} \mu_{ax+b} &= a\mu_x + b \\ \sigma^2_{ax+b} &= a^2\sigma^2_x \end{aligned} \tag{7.8}$$

These both follow from (7.7) and the definitions of mean and variance in (7.5) and (7.6), and they are the subject of Exercise 1 at the end of this chapter.

7.3 Uniform, Gaussian, and Other Distributions

Turning now to specific examples of amplitude distributions, two distributions are of particular interest in digital signal analysis. First is the *uniform* distribution illustrated in Figure 7.4. The uniform probability density function expresses the assumption that all values of x in the interval $[a,b]$ are "equally likely." (This notion has real meaning only in the discrete case;

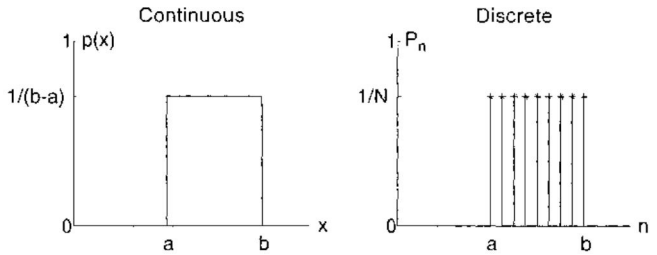

FIGURE 7.4
Continuous and discrete uniform amplitude distribution functions.

nevertheless, it has heuristic appeal in the continuous case.) In the continuous case, the sample space is the horizontal (x) axis, and the integral of $p(x)$ satisfies (7.1). In the discrete case, the sample space consists of N integer values of x in the range $[a,b]$, and the sum of P_n over n again satisfies (7.1). In either case, if the amplitudes of $x(t)$ and its samples are uniformly distributed, $x(t)$ is called a *uniform variate*.

Using the continuous probability function, the *mean value* of the uniform variate in Figure 7.4 is found as follows:

$$\mu_x = \int_{-\infty}^{\infty} xp(x)dx = \frac{1}{b-a}\int_a^b x\,dx = \frac{b+a}{2} \tag{7.9}$$

Using the result in (7.6), the *variance* of the uniform variate is

$$\sigma_x^2 = E[x^2] - \mu_x^2 = \frac{1}{b-a}\int_a^b x^2 dx - \frac{(b+a)^2}{4} = \frac{(b-a)^2}{12} \tag{7.10}$$

Thus, the *standard deviation*, σ_x, is $1/\sqrt{12}$ times the range of values of x, that is, $b-a$.

The second important example of an amplitude distribution is the *Gaussian* or *normal* probability function, illustrated in Figure 7.5. This function is of interest in DSP primarily, because, with many types of data (electromagnetic, acoustic, seismic, vibration, etc.), the amplitude distributions of random noise are Gaussian, or approximately so. The general form of the Gaussian probability density function is

$$p(x) = N(\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{7.11}$$

In this expression, μ is the mean value, μ_x, and σ is the standard deviation, σ_x. The discrete form, shown on the right in Figure 7.5, is a discrete distribution function, P_n, having an envelope given by (7.11) and satisfying the condition (7.1).

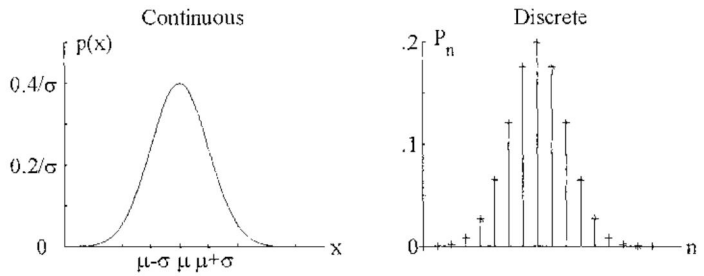

FIGURE 7.5
Continuous and discrete Gaussian amplitude distribution functions.

To see that $N(\mu, \sigma)$ has a unit integral as in (7.1), we may substitute $y = (x - \mu)/(\sigma\sqrt{2})$, so the integral of $p(x)$ becomes

$$\int_{-\infty}^{\infty} p(x)dx = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-(x-\mu)^2/2\sigma^2} dx = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} e^{-y^2} dy \tag{7.12}$$

In this form, the integral can be evaluated simply by evaluating its square with the use of polar coordinates:

$$\left[\int_{-\infty}^{\infty} p(x)\,dx\right]^2 = \frac{1}{\pi}\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{-y^2} e^{-v^2} dy\,dv = \frac{1}{\pi}\int_0^{2\pi}\int_0^{\infty} e^{-r^2} r\,dr\,d\theta = 2\int_0^{\infty} re^{-r^2} dr = 1 \tag{7.13}$$

The demonstration that μ is the mean value of x is made by placing the Gaussian function into the definition (7.5) and again making the substitution in (7.12) for x, that is, $x = \sigma y\sqrt{2} + \mu$:

$$\begin{aligned}
\mu_x &= \int_{-\infty}^{\infty} \frac{x}{\sigma\sqrt{2\pi}} e^{\frac{(x-\mu)^2}{2\sigma^2}} dx = \int_{-\infty}^{\infty} \frac{\sigma y\sqrt{2}}{\sqrt{\pi}} e^{-y^2} dy + \frac{\mu}{\sqrt{\pi}} \int_{-\infty}^{\infty} e^{-y^2} dy \\
&= \frac{\sigma\sqrt{2}}{\sqrt{\pi}} \int_{-\infty}^{\infty} ye^{-y^2} dy + \frac{\mu}{\pi} \int_{-\infty}^{\infty} e^{-y^2} dy \\
&= 0 + \mu = \mu
\end{aligned} \tag{7.14}$$

Thus, the use of μ in (7.11) is justified. Similarly, for the variance as given in (7.6), we can let $\mu = 0$ and obtain σ_x^2 as the expected value of x^2 as follows, doing the integration by parts (Chapter 1, Table 1.3):

$$\sigma_x^2 = E[x^2] = \int_{-\infty}^{\infty} \frac{x^2}{\sigma\sqrt{2\pi}} e^{\frac{-x^2}{2\sigma^2}} dx = \sigma^2 \tag{7.15}$$

Thus, because $\mu = \mu_x$, σ^2 is the variance of the Gaussian distribution.

Unfortunately, when the distribution of $x(t)$ is given by $N(\mu, \sigma)$ as in (7.11), the probability that x lies between some pair of values [as in (7.2)] cannot be easily expressed as it obviously can be, for example, in the case of the uniform distribution. Therefore, normalized versions of this probability, which is called the *error function* (*erf*) are approximated in various ways. In MATLAB, the computation is done by *erf*(v), which computes the *normalized error function* of each element of a vector v as follows:

$$\text{erf}(v) = \frac{2}{\sqrt{\pi}} \int_0^v e^{-y^2} dy. \tag{7.16}$$

To apply erf(v) to the form of $p(x)$ shown in (7.11) and in Figure 7.5, we again use the substitution used to obtain (7.12), in this case, $y = (x - \mu)/\sigma\sqrt{2}$:

$$\text{erf}(v) = \frac{2}{\sqrt{\pi}} \int_{\mu}^{\mu + v\sigma\sqrt{2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \frac{dx}{\sigma\sqrt{2}} = 2\int_{\mu}^{\mu + v\sigma\sqrt{2}} N(\mu, \sigma) dx; \quad \text{or,}$$
$$\int_{\mu}^{\mu + x} N(\mu, \sigma) dx = \frac{1}{2} \text{erf}\left(\frac{x}{\sigma\sqrt{2}}\right) \tag{7.17}$$

Assuming $x \geq 0$, the second result gives us the probability that a normal variate lies in the range $(\mu, \mu + x)$ in terms of the MATLAB function, *erf*.

An example is shown in Figure 7.6, which illustrates the approximately Gaussian amplitude distribution $N(2000, 600)$ of a signal (x) from a 12-bit analog-to-digital converter. The distribution of x is discrete, but with $2^{12} = 4096$ possible values, it may be viewed as continuous. In this case, because the values of x are separated by unit distance, the integral of $p(x)$ and the sum of discrete probabilities (P_n) must be the same. To compute

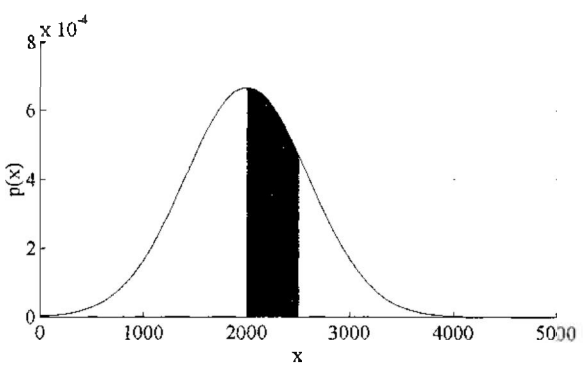

FIGURE 7.6
Amplitude distribution of signal sampled with a 12-bit A-D converter. The dark area, which is $Pr\{2000 < x < 2500\}$, is computed as shown in (7.17) and the text.

the dark area shown in the figure, we would use (in MATLAB notation) *darkarea* = .5 * *erf*((2500-*mu*)/(*sigma* * *sqrt*(2))) with *mu* = 2000 and *sigma* = 600.

Two MATLAB functions are used to produce random sequences and arrays. (Other computing languages have similar functions.) *rand*(*M*,*N*) produces an *M* by *N* array of uniform random variates with each element in the range (0,1). Similarly, *randn*(*M*,*N*) produces an array of Gaussian variates from the distribution *N*(0,1). These functions actually generate *pseudo-random* sequences and arrays using number-generating algorithms with very long cycles. To use the same random sequence, say in a test or simulation, the expressions *rand*('*seed*',*K*) and *randn*('*seed*',*K*) may be used to reset the starting point.

Given (7.8), which allows us to make a linear treansformation of any random variate, we can deduce the following rules for generating zero-mean random sequences with variance σ^2:

Random vectors: length = N, mean = 0, and standard deviation = σ	
Uniform: $x = \sigma * \sqrt{12} * (rand(1,N) - 0.5)$	(7.18)
Gaussian: $x = \sigma * randn(1,N)$	

Two examples are shown in Figure 7.7. In each case, N = 10,000 samples, and each element of x is rounded to the nearest integer to simulate the output of a digital device such as an analog-to-digital converter. On the left is the frequency function of a uniform sequence with σ = 4.33, which translates

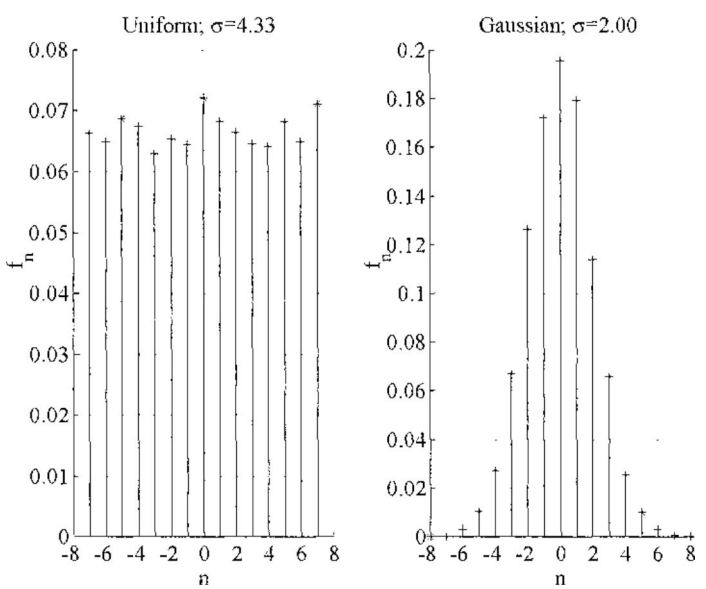

FIGURE 7.7
Amplitude distributions of digitized uniform and Gaussian random sequences. In each case, N = 10,000 samples, μ = 0, and σ is shown in the figure.

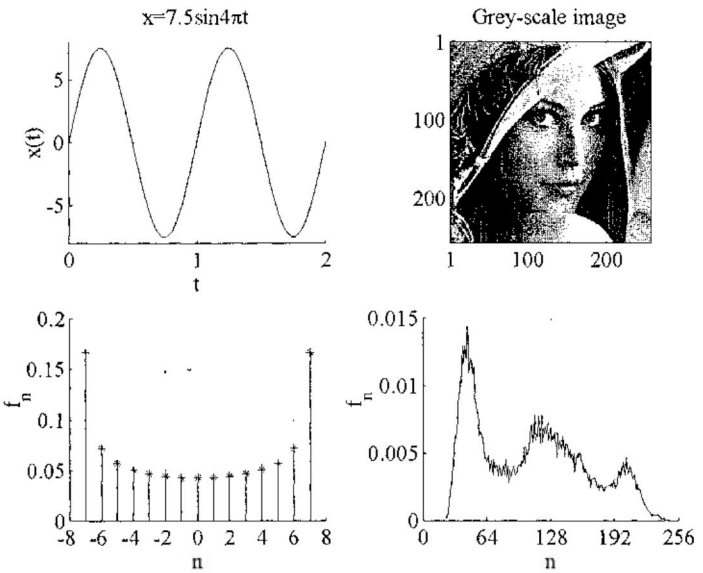

FIGURE 7.8
Amplitude distributions that are neither uniform nor Gaussian. The amplitude distribution of a digitized sine wave is on the left, and the distribution of gray-scale image pixel values is on the right.

via (7.18) to equally likely integer values in the range $[-7, 7]$. On the right is the frequency function of a Gaussian sequence with $\sigma = 2$. In both cases, we note that, due to the finite value of N, the distributions are only approximately the same as the theoretical distributions.

Obviously, a sequence does not need to be random or even have random components in order to have an amplitude distribution. Essentially all sequences and arrays of samples have amplitude distributions. And although the amplitude distribution is used mainly in the analysis of random functions, it is also used with all types of functions in the analysis of coding and compression techniques, which are important areas of DSP discussed in Chapter 10. So to conclude our discussion here, we offer also Figure 7.8, which consists of two nonrandom examples, both with $N = 2^{16}$ samples. On the left is a sine wave with amplitude 7.5, with samples rounded to the nearest integer before compiling the amplitude distribution, $[f_n]$. Note how the amplitude distribution is symmetric and indicates that values of $f(t)$ near the minimum and maximum are more likely than values near zero. The amplitude distribution of a continuous sine wave $y(x)$ may be found using (7.7), beginning with a uniform distribution of x and transforming $p(x)$ into $p(y)$. See Exercise 2 at the end of this chapter.

On the right in Figure 7.8 is the image known as "Lena," which has been a standard gray-scale image in DSP literature for decades at the time of this publication. (We pause here to thank the subject for her notable contribution

to image processing and hope the years have been good to her.) In this version, the image is 256 × 256 pixels, and each pixel value is on a black-to-white scale from 0 to 255 (although not all pixel values are present in the image). The amplitude distribution in this case shows the relative frequencies of different pixel values occurring in the image. In both cases, the frequency functions sum to one in accordance with (7.1) and (7.3).

7.4 Power and Power Density Spectra

In Chapter 4, Section 4.13, we measured the power gain of a linear system in terms of the square of the transfer function magnitude, which in turn is measured by the DFT of the impulse response. Thus, we could say that the square of the DFT magnitude of any function $x(t)$ is a measure of the distribution of the power in $x(t)$ over the frequency domain. Now we wish to pursue this idea and arrive at precise definitions of *power* and *power density*.

The *instantaneous power* of $x(t)$ at any time t is the squared signal magnitude, $|x(t)|^2$. We assume the signal is real so that $x^2(t)$ may be used, but the general definition holds even for complex signals. It follows that the *average* or *expected power* of a stationary function $x(t)$ is

$$\text{Average power} = E[x^2(t)] = \lim_{T\to\infty} \frac{1}{T}\int_{-T/2}^{T/2} x^2(t)dt \tag{7.19}$$

If x is a vector consisting of N samples of $x(t)$, we say the average power in x is

$$\text{Average power} = \frac{1}{N}\sum_{n=0}^{N-1} x_n^2 \approx \frac{1}{NT}\int_0^{NT} x^2(t)dt \tag{7.20}$$

The average power in this sense is an *estimate* of the true average power in (7.19).

To express the average power in terms of the DFT, we substitute the inverse DFT formula in (3.23) for x_n in (7.20) and write the result as follows:

$$\text{Avg. power} = \frac{1}{N}\sum_{n=0}^{N-1}\left[\frac{1}{N}\sum_{m=0}^{N-1} X_m e^{j2\pi mn/N}\right]^2 = \frac{1}{N^3}\sum_{m=0}^{N-1}\sum_{k=0}^{N-1} X_m X_k \sum_{n=0}^{N-1} e^{j2\pi(m+k)n/N} \tag{7.21}$$

In this result, we may substitute $k = N - i$, use the redundancy in (3.10) to note that $X_0 = X_N$, and use the redundancy in (3.11) to let $X_{N-i} = X_i'$. Then the average power expression becomes

$$\text{Avg. power} = \frac{1}{N^3}\sum_{m=0}^{N-1}\sum_{i=0}^{N-1} X_m X_i' \sum_{n=0}^{N-1} e^{j2\pi(m-i)n/N} = \frac{1}{N^2}\sum_{m=0}^{N-1} |X_m|^2 \tag{7.22}$$

The result on the right follows, because the inner sum in the center equals N when $m = i$ and is zero otherwise. Thus, in combination with (7.20), we have the result

$$\boxed{\sum_{n=0}^{N-1} x_n^2 = \frac{1}{N}\sum_{m=0}^{N-1} |X_m|^2} \tag{7.23}$$

This result presents average signal power in terms of the power spectrum. Known as *Parseval's theorem*, it provides an important insight and link between the time and frequency domains.

Our final step in the development of the notion of a *power density spectrum* is to define the *periodogram*. The periodogram has N components given by

$$\boxed{\text{Periodogram:} \quad P_{xx}(m) = \frac{1}{N}|X_m|^2; \quad m = 0, 1, \ldots, N-1} \tag{7.24}$$

The periodogram is thus a real function of m, periodic in accordance with (3.10) and with $P_{xx}(N - m) = P_{xx}(m)$ in accordance with (3.11). Furthermore, using the results obtained above, we may write

$$\text{Average power} = \frac{1}{N}\sum_{n=0}^{N-1} x_n^2 = \frac{1}{N}\sum_{m=0}^{N-1} P_{xx}(m) \tag{7.25}$$

Thus, in the same way that x_n^2 gives a measure of signal power at a point in the time domain, $P_{xx}(m)$ gives a measure of signal power at a point in the frequency domain. Therefore, the vector $P_{xx} = [P_{xx}(m)]$ is said to be a measure of the *power density spectrum* of $x(t)$, because its average value in (7.25) is another expression of the average power estimate.

Power density in this form is best applied to stationary signals and images from which segments are extracted for analysis, as described in the next section. If a signal component is considered to have a definite beginning and end, and if the sampling process covers the entire range, then we use *energy* measures instead of power. Energy is the integral of power over time or space. In terms of samples, the measure of signal energy is

$$\text{Energy} = T\sum_{m=0}^{N-1} P_{xx}(m) = T\sum_{n=0}^{N-1} x_n^2 \approx \int_0^{NT} x^2(t)dt \tag{7.26}$$

Thus, the periodogram *average* is our measure of signal power, and the *total* periodogram is our measure of signal energy.

Examples of power and energy spectra in terms of the periodogram are shown in Figures 7.9 and 7.10. Figure 7.9 shows one cycle of a periodic

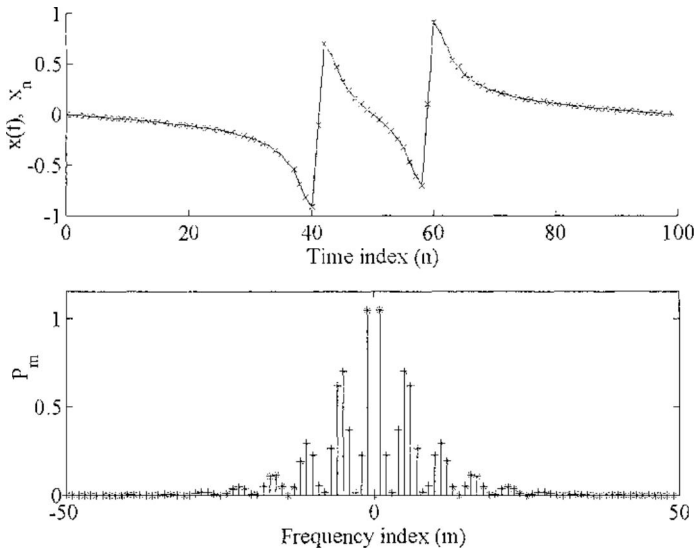

FIGURE 7.9
A sampled periodic signal and its periodogram. One hundred samples of a single cycle of $x(t)$ are shown in the upper plot, and the periodogram is shown below.

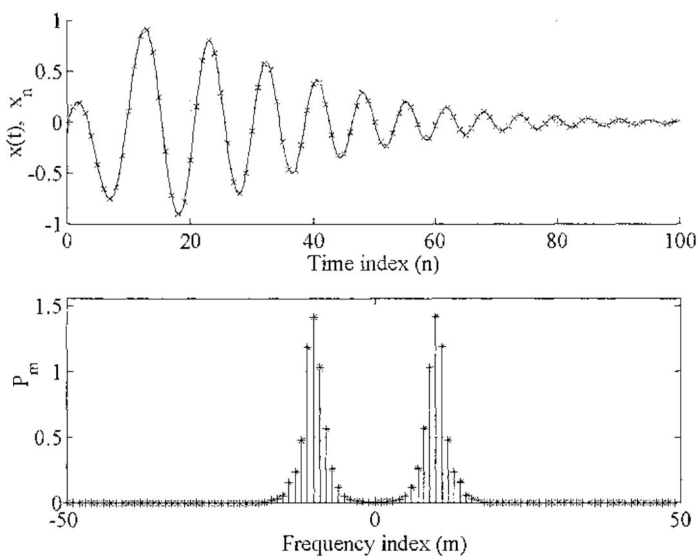

FIGURE 7.10
A transient signal and its periodogram. One hundred samples of $x(t)$ are shown in the upper plot, and the periodogram is shown below.

function and its periodogram. The latter represents power density in the sense that (7.25) holds, with the average power being about 0.09 in this example. Figure 7.10 shows a transient signal and its periodogram. The periodogram in this case represents energy density in the sense that (7.26) holds, with the energy being approximately 11.3 in the example, assuming $T = 1$. The two periodograms are computed in the same manner, but there is a difference in how they are used to represent power or energy. To make the discussion easier, from here on, we will discuss the periodogram as representing *power density*, because this is how it is used with random signals.

When $x(t)$ represents a time-varying physical quantity, the power spectrum is sometimes plotted vs. the frequency index as it is in Figure 7.9, but more often, it is plotted in terms of power per unit of frequency—usually *power per Hz* or *power per Hz-s*. In this case, a *bar graph* is used in place of the discrete plot in Figure 7.9. The bar graph is made such that, in the power spectrum,

$$\begin{aligned}\text{Power in range } (|m| \pm 0.5)/N \text{ Hz-s} &= (\text{area of bar at } m) + (\text{area of bar at } -m) \\ &= 2(\text{area of bar at } m)\end{aligned} \tag{7.27}$$

(and similarly for the energy spectrum). Figure 7.11 shows the periodogram in Figure 7.9 modified in this manner. That is, the power represented by each line in Figure 7.9 is represented by the area of the corresponding bar in Figure 7.11. Thus, the total power, or average power, is the integral of the power spectrum, and (7.25) holds in either case, because the bar width in Figure 7.11 is $1/N$. Note, however, that if the frequency units are changed from Hz-s to Hz, the bar width is now $1/NT$, and so the power density must

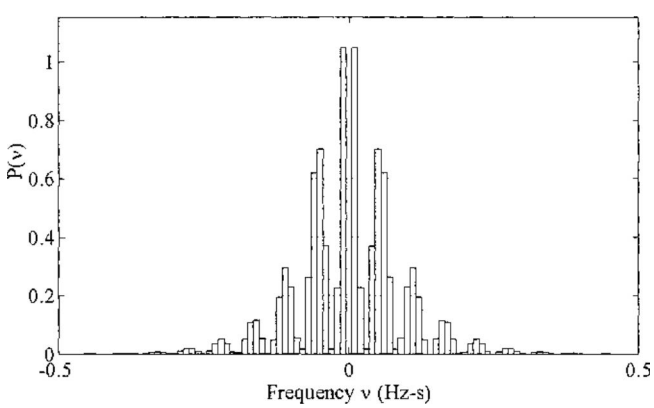

FIGURE 7.11
Periodogram in Figure 7.9 redrawn as a power density spectrum. A bar centered over $\nu = m/N$ represents power in the range $(m \pm 0.5)/N$, and the integral of the entire spectrum is equal to the average squared signal value.

be scaled by T for the integral to agree with (7.25). The general version of (7.25) accommodating different familiar frequency notations is

$$\text{Average power} = \frac{1}{N}\sum_{n=0}^{N-1} x_n^2 = \frac{1}{N}\sum_{m=0}^{N-1} P_{xx}(m) = \int_{-0.5}^{0.5} P_{xx}(\nu)\, d\nu$$
$$= \int_{-1/2T}^{1/2T} P_{xx}(f)\, df = \int_{-\pi/T}^{\pi/T} P_{xx}(\omega)\, d\omega \tag{7.28}$$

To see how each spectral measure must be scaled, assume $P_{xx}(m)$ is constant over m and equal to P. Then we can see that $P_{xx}(\nu) = P$, $P_{xx}(f) = TP$, and $P_{xx}(\omega) = TP/2\pi$ will all produce the same average power, that is, P.

7.5 Properties of the Power Spectrum

There are many examples in DSP where the source of a signal, $x(t)$, is such that $x(t)$ is stationary over a long period and contains random components. The power spectrum, estimated from recorded segments of $x(t)$, often reveals facts about the source that we would not notice in the raw data. The power spectrum and the amplitude distribution together contain the basic signal statistics that are normally required for analysis, filtering, signal compression, and other types of processing. In the next section, we discuss a method for estimating the power spectrum of a stationary random sequence. Here we review its properties, the most important being its relationship to the autocorrelation function.

Suppose a signal vector x is acquired in accordance with Figure 7.12. In practice, aliasing may be prevented by analog prefiltering, or simply by sampling at a rate greater than twice the maximum frequency response of the sensor. The frequency-limited signal, $x(t)$, is sampled to produce the vector x consisting of integer samples. For the following analysis, we assume a periodic extension of x, which is reasonable when the signal is stationary. Then, in accordance with (3.2) in Chapter 3, the autocorrelation function of

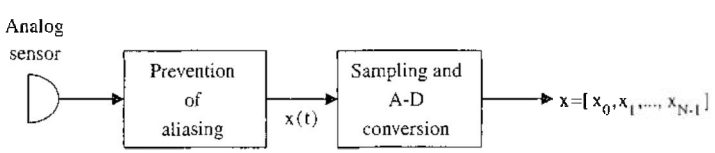

FIGURE 7.12
Acquiring the sample vector, x, from the measurement of a continuous signal.

x (extended periodically) is

$$\varphi_{xx}(k) = \frac{1}{N}\sum_{n=0}^{N-1} x_n x_{n+k}; \quad k = 0, 1, \ldots, N-1 \tag{7.29}$$

From this definition and (7.25), we notice that $\varphi_{xx}(k)$ is also periodic, and furthermore,

$$\text{Average power} = \frac{1}{N}\sum_{n=0}^{N-1} x_n^2 = \varphi_{xx}(0) \tag{7.30}$$

We can show by the following that the periodogram is the DFT of $\varphi_{xx}(k)$. First, using the definition (3.7) of the DFT,

$$\text{DFT}\{\varphi_{xx}(k)\} = \frac{1}{N}\sum_{k=0}^{N-1}\sum_{n=0}^{N-1} x_n x_{n+k} e^{-j2\pi km/N}; \quad m = 0, 1, \ldots, N-1 \tag{7.31}$$

Now substitute the inverse DFT (3.23) for x_n and x_{n+k}, using the linear phase shift property (Table 3.3, Property 8) on the latter (the range of m remains the same throughout):

$$DFT\{\varphi_{xx}(k)\} = \frac{1}{N^3}\sum_{k=0}^{N-1}\sum_{n=0}^{N-1}\sum_{\alpha=0}^{N-1} X_\alpha e^{j2\pi\alpha n/N}\sum_{\beta=0}^{N-1} X_\beta e^{j2\pi\beta(n+k)/N} e^{-j2\pi km/N}$$

$$= \frac{1}{N^3}\sum_{\alpha=0}^{N-1}\sum_{\beta=0}^{N-1} X_\alpha X_\beta \sum_{k=0}^{N-1}\sum_{n=0}^{N-1} e^{j2\pi[(\alpha+\beta)+(\beta-m)k]/N} \tag{7.32}$$

Into this equation, we substitute $\alpha = N - \gamma$. The reduction is then similar to the reduction of (7.21), and we obtain the final result:

$$\text{DFT}\{\varphi_{xx}(k)\} = \frac{1}{N^3}\sum_{\beta=0}^{N-1}\sum_{\gamma=0}^{N-1} X_\beta X_\gamma' \sum_{k=0}^{N-1} e^{j2\pi(\beta-m)k/N}\sum_{n=0}^{N-1} e^{j2\pi(\beta-\gamma)n/N}$$

$$= \frac{1}{N}|X_m|^2 = P_{xx}(m); \quad m = 0, 1, \ldots, N-1 \tag{7.33}$$

The final result follows because, similar to (7.22), the sums over k and n are nonzero and equal to N only when $\alpha = \gamma = m$. Thus, we have the following important result:

> The periodogram of a vector x with periodic extension is the DFT of the autocorrelation function of x; that is,
> $\text{DFT}\{\varphi_{xx}(k)\} = P_{xx}(m); \quad m = 0, 1, \ldots, N-1$ (7.34)

Our usual view of the autocorrelation function is that φ_{xx} gives us information on the dependence of a given sample, x_n, on nearby samples, $x_{n\pm1}$,

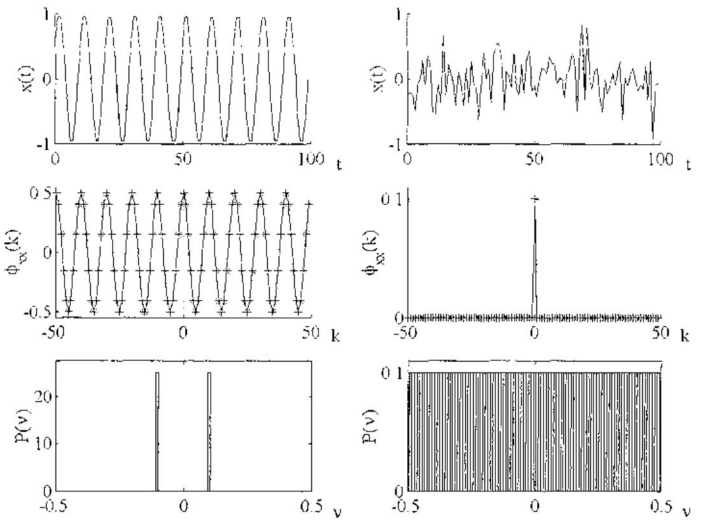

FIGURE 7.13
Signals, autocorrelation functions, and periodograms for a unit sinusoid (left) and white Gaussian noise (right). The average power, $\varphi_{xx}(0)$, is 0.5 for the sinusoid and 0.1 for the noise.

$x_{n\pm2}$, and so on. The periodogram, as we have seen, tells us the distribution of signal power, x_n^2, over frequency. Thus, these two kinds of information about the signal are equivalent via (7.34). Notice also that neither φ_{xx} nor P_{xx} contain any phase information about x; that is, the functions are not affected by shifting x in the time domain.

Two special examples of (7.34) are worth mentioning. First is the case where x is any sinusoidal component of a waveform, sampled over K cycles. It is easy to show in this case that $\varphi_{xx}(k)$ is a cosine function, regardless of the phase of x, and $P_{xx}(m)$ is zero everywhere except at $m = \pm K$. In the second case, x is a segment of *white noise*, that is, a random signal with the same power density at all frequencies. In this case, $\varphi_{xx}(k)$ is an impulse at $k = 0$, showing that the signal elements are statistically independent. Both of these cases are illustrated in Figure 7.13.

On the left in Figure 7.13, $x(t)$ is sampled with $T = 1$. The autocorrelation function $\varphi_{xx}(k)$, plotted below $x(t)$, is a cosine function. Note that $\varphi_{xx}(0) = 0.5$, which is the average squared value of $x(t)$. The power spectrum, plotted at the bottom, shows that all the power is at $v = 0.1$, which is the frequency of $x(t)$ in Hz-s. The integral of the power spectrum, in accordance with (7.28), is the average power, or 0.5.

On the right in Figure 7.13, the random signal $x(t)$ is again sampled with $T = 1$. The functions $\varphi_{xx}(k)$ and $P_{xx}(v)$ are the statistical properties of the stationary signal from which the segment $x(t)$ was taken, not the measured properties of $x(t)$, in order to better illustrate the properties of white noise.

7.6 Power Spectral Estimation

As in the preceding section, let us assume that $x(t)$ is stationary and contains random components. Let us also assume that we have a recording of $x(t)$ in terms of a long sample vector obtained as in Figure 7.12. From this vector, we wish to estimate the power spectrum of $x(t)$. There are many applications like this in statistical analysis, where the characteristics of a population are estimated using samples taken from the population.

Having the sample vector, intuition would suggest at first that we use its periodogram as the estimate of the power spectrum, but this approach turns out to be not useful. The reason has to do with the number of degrees of freedom (and thus the variance) of the periodogram [as if we had a large collection of periodograms of $x(t)$, and could measure the variance of each component, $P_{xx}(m)$]. Because, by definition, $P_{xx}(m)$ is found from the DFT magnitude, the periodogram must always have $N/2$ degrees of freedom. Therefore, the degrees of freedom of the periodogram increase linearly with the segment length, and the periodogram of a long sample vector taken from a random function, $x(t)$, is essentially no more reliable than the periodogram of a short sample vector as an estimate of the true power spectrum.

This concept is illustrated in the two upper power spectra in Figure 7.14. (Here, we plot only the positive half of the spectra, using connected points

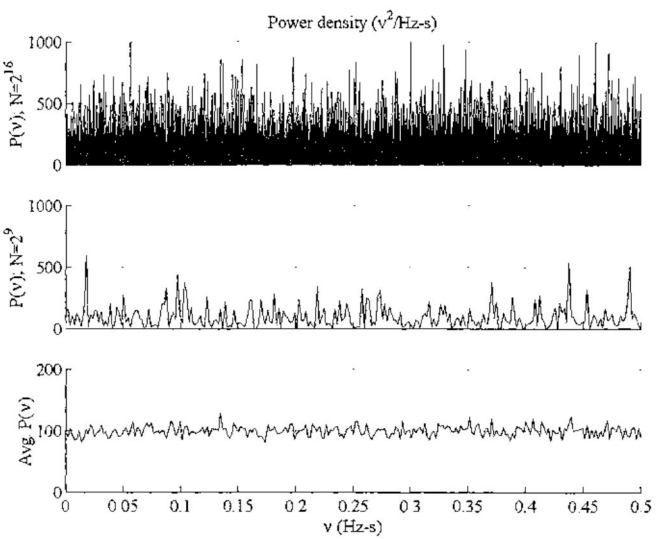

FIGURE 7.14
Power spectral estimates. The first two, based on single periodograms, are not good estimates. The third is the average of 128 periodograms and is a good estimate of the true power density, which is 100 at all frequencies.

instead of bars on account of the large values of N.) Referring to Figure 7.12, for this illustration, we simulated the sampling, 12-bit conversion, and scaling to units of volts (v) of a white random signal with average power equal to 100 v^2. The upper periodogram is based on 2^{16} samples, and the center periodogram on 2^9 samples of the signal. Both periodograms are essentially useless estimators of the true power spectrum.

On the other hand, the periodogram at the bottom of Figure 7.14 gives an estimate of the true power spectrum that is quite accurate. It was constructed by partitioning the same segment with 2^{16} samples into 2^7 successive, nonoverlapping segments, each of length 2^9, and by then averaging these to produce an *average periodogram*. We can see in this case that the power density is approximately 100 v^2/Hz-s at all frequencies.

Thus, if we have a recorded segment of a stationary function with random components, our method for obtaining a useful estimate of the power spectrum consists of partitioning the recorded segment into smaller segments and averaging the periodograms of the smaller segments. It even helps to *overlap* the smaller segments. At first, it may appear that overlapping does not improve the estimate, because samples are re-used. But although the samples are re-used, overlapping improves the estimate, because it allows more segments in a given recording, and even though the segments overlap, each segment is different.

This partitioning concept is illustrated in Figure 7.15. At the top, $x(kT)$ is a plot of $N = 1024$ samples of a stationary random signal. The segments x_1

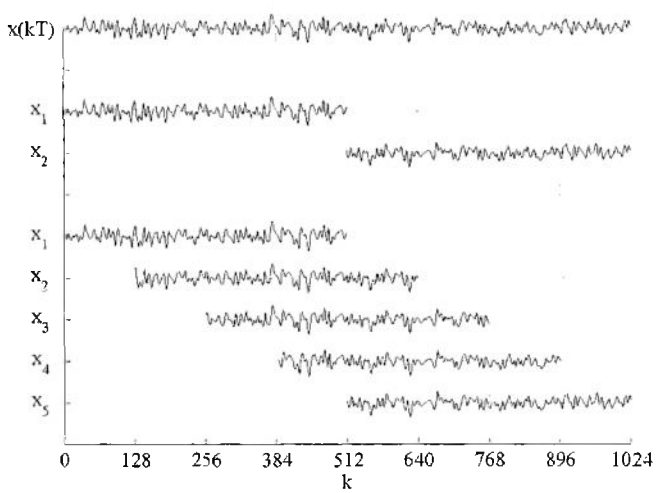

FIGURE 7.15
Partitioning $N = 1024$ samples of a recorded random waveform into segments of length $N/2 = 512$. Segments x_1–x_2 illustrate two nonoverlapping segments, and segments x_1–x_5 illustrate five segments with 75% overlap.

and x_2 in the middle of the figure, each with 512 samples, are obtained simply by partitioning x into two halves. The segments x_1 through x_5 at the bottom, each again with 512 samples, are the result of applying a rectangular, 512-sample window to x, with the window shifting 128 samples for each successive segment. Thus, the figure represents three choices for power spectral estimation using the same recorded data, and these choices may be rated as follows:

Worst: Periodogram of x
Better: Average periodogram of two independent segments
Best: Average periodogram of five overlapping segments.

Even though the overlapping segments contain redundant data, no two segments are alike, and a larger averaged number of periodograms means a better estimate.

The *average periodogram method* for estimating the power spectrum of a stationary waveform has been described by Welch.[16] Welch suggested using half-overlapping segments. One reason for this choice is that more overlap, although it helps, does not produce proportionately less variance in the spectral estimate, and half-overlapping is a good compromise. A second reason is that an FFT is required for each segment, so more overlap translates into more computing. However, the second reason is not as important as it was in the "old days" when computers were orders of magnitude slower than they are now. More overlap, even to the point where the window moves only one sample at a time, generally produces a better spectral estimate. The accuracy of spectral estimation is discussed in the literature.[17–21]

This principle is illustrated using $N = 1024$ samples of filtered white Gaussian noise, generated as shown in Figure 7.16. The input signal vector, x, is a record of white Gaussian noise with zero mean and unit standard deviation, in other words, a recording from a signal having unit power and unit power density in accordance with (7.6). The true power spectrum of the bandpass filter output, y, is therefore the same as the power gain of the filter.

Three power spectral computations are shown in Figure 7.17. The true bandpass power spectrum is plotted as the heavy line in each case. Each spectral

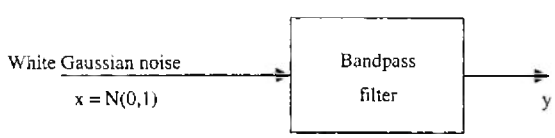

FIGURE 7.16
Illustration showing how the signal (y) is produced for the examples of power spectra in Figure 7.17. The input (x) is white Gaussian noise with $\mu = 0$ and $\sigma = 1$, that is, unit power density. The power spectrum of y is therefore the squared amplitude gain of the bandpass filter.

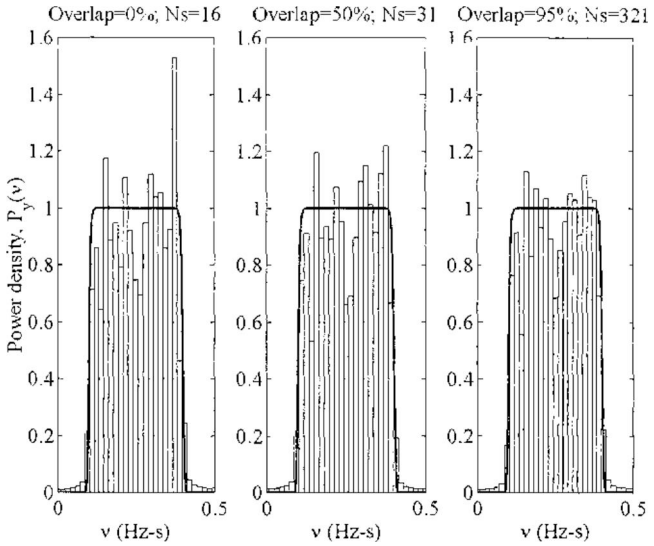

FIGURE 7.17
Power density, $P_y(\nu)$, computed from 1024 samples of the signal y in Figure 7.16. In each plot, the segment size was 64 samples, and the window was rectangular. The segment overlap, and hence, the number of segments (Ns), is the only change from one spectrum to the next. The heavy line in each case shows the true power spectrum.

computation was made with the same record (y) having length $N = 1024$, using 64-sample segments and a rectangular data window. Only the overlap is changed, and we can see that the accuracy seems to improve with the degree of overlap. But we also note that the number of segments, and thus the number of FFTs, only approximately doubles as we go to 50% overlap, whereas this number increases by another order of magnitude as we go to 95% overlap. Thus, the choice of overlap becomes a matter of judgment, depending on the factors we mentioned.

We should also note that the foregoing is based on the idea of getting the most we can out of a fixed signal record of fixed length, such as the recording from an instrument, or telemetry, of a source that has finite duration. If an unlimited length of the stationary random signal is available, then it is always preferable to use independent, nonoverlapping segments.

Another matter of judgment concerns the selection of the segment size, again assuming a recording of fixed length. Obviously, a shorter segment length means more segments and a more accurate spectral estimate. On the other hand, if the segment length is K, then the frequency resolution is $1/K$ Hz-s, and so a shorter segment length also means less resolution. On account of this tradeoff, the value of K becomes a matter of choice. The effect of reducing K in order to get a better spectral estimate is illustrated in Figure 7.18, which again is produced using a signal y as produced in Figure 7.16. In this

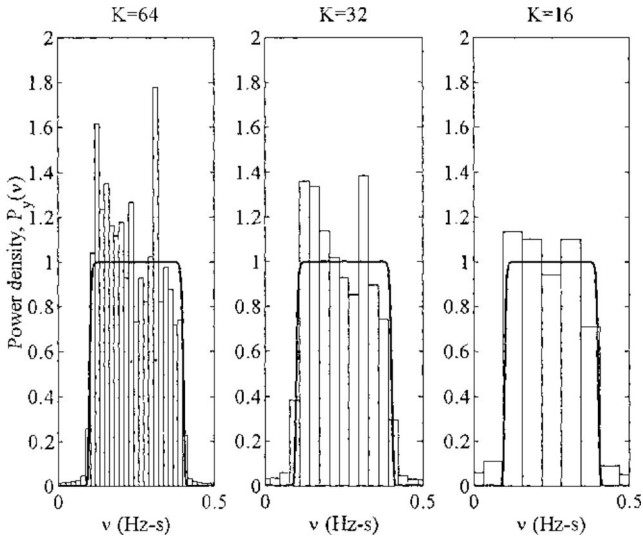

FIGURE 7.18
Power density, $P_y(\nu)$, computed from 1024 samples of the signal y in Figure 7.16. In each plot, the segment overlap was 95%, and the window was rectangular. The segment size (K) is the only change from one spectrum to the next. The heavy line in each case shows the true power spectrum.

example, the overlap was 95% in all three cases, K was reduced from 64 to 32 to 16, and we can see the improvement in the spectral estimate.

The illustrations in Figures 7.17 and 7.18 are random in the sense that examples like these will be quite different for different random sequences due to the short record length of only $N = 2^{10}$ samples (compared with 2^{16} samples used for the lower plot in Figure 7.14). In other words, the examples in Figures 7.17 and 7.18 illustrate the concepts of improving spectral estimates, but they are not "typical" in any other sense.

7.7 Data Windows in Spectral Estimation

In Chapter 5, Section 5.4, we used window functions to modify the impulse response of an ideal FIR filter in order to obtain a smoothed version of the filter's power gain function. In the same way, the application of a window function to a random signal segment should, in general, result in a smoothed periodogram. Windows are often used for this purpose in spectral estimation.

The effect of the data window is governed by the relationship (5.7), which states in effect that when any signal vector is multiplied by a window vector, the spectral result is the periodic convolution of the DFTs of the two vectors.

That is,

$$\mathrm{DFT}\{w_k x_k\} = \frac{1}{N}\sum_{n=0}^{N-1} W_n X_{m-n}; \quad m = 0, 1, \ldots, N-1 \tag{7.35}$$

where w and x are the window and signal vectors of length N. The convolution of the window spectrum with the signal spectrum has the effect shown in the examples of Chapter 5. In the present case, it improves the spectral estimate by smoothing it, but also smears abrupt changes in the estimated power density, making the estimated spectrum somewhat less accurate in this respect.

When we use a window vector (w) to modify a data segment (x) in power spectral estimation, it is also important to remember that the window alters the total squared value of the data. That is, unless we are using the boxcar window, $(w .* x) .\hat{}2$ is not equal to $x .\hat{}2$. Therefore, to preserve (7.28) and have the integral of the periodogram equal to the average power, we must remove this effect of the window by dividing each periodogram component by $w * w'$, that is, we must divide each periodogram by the total squared window value.

An illustration of the use of data windows (scaled as just described) is given in Figure 7.19, which again shows power spectral estimates of the signal y generated as in Figure 7.16. In this illustration, the record size was increased to 2^{12} samples to give a better estimate for segment length 64. The

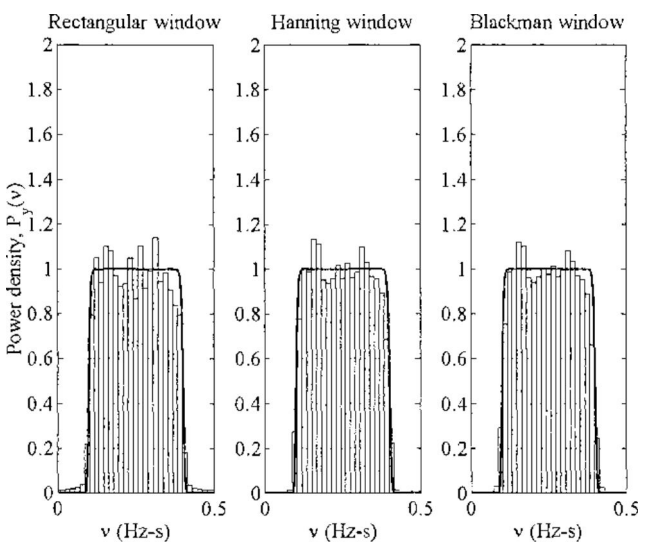

FIGURE 7.19
Power density, $P_y(\nu)$, computed from 4096 samples of the signal y in Figure 7.16. In each plot, the segment overlap was 95%, and the segment size was 64. The plots show the effect of using different data windows. The heavy line in each case shows the true power spectrum.

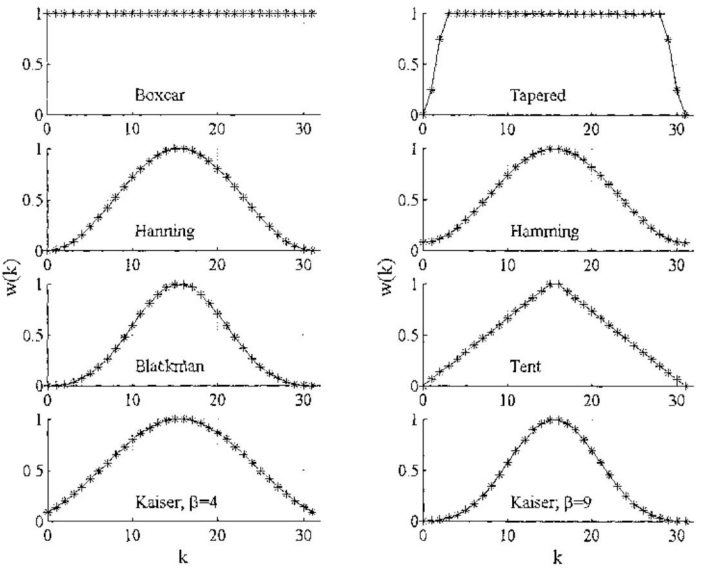

FIGURE 7.20
Data windows generated by the *windows* function; $N = 32$.

overlap was again set at 95%. We can note that the Hanning and Blackman windows produce smoothing along the top of the spectrum, but (on close inspection) they have the adverse effect of causing the edges at 0.1 and 0.4 Hz-s to be less defined.

The use of windows in power spectral estimation is generally recommended with stationary random signals, especially signals with broad spectra. The windows generated by the function *window*, described in Chapter 5, (5.13), are illustrated in Figure 7.20. Any of these may be used to smooth the periodograms that are averaged to obtain an estimated power spectrum.

If a signal is not stationary or has a true power spectrum with sharp discontinuities, data windows must be used with caution. In this case, the *boxcar* and the *tapered rectangular* window, which are among the choices illustrated in Figure 7.20, are generally preferable. The tapered rectangular window is almost rectangular, having a cosine taper over 10% of the length at each end.

7.8 The Cross-Power Spectrum

The *cross-power spectrum*, or *cross spectrum* for short, is like the power spectrum we have been discussing, except it involves two time series instead of just one. Suppose x and y are vectors of length N taken from two waveforms, $x(t)$ and $y(t)$. As before, we assume for this analysis that both x and

y extend periodically. Then, analogous to the periodogram in (7.24), we have the *cross-periodogram*:

$$\text{Cross-periodogram:}\quad P_{xy}(m) = \frac{1}{N}X'_m Y_m; \quad m = 0, 1, \ldots, N-1 \tag{7.36}$$

Just as $P_{xx}(m)$ was a measure of power density, so also $P_{xy}(m)$ is a measure of *cross-power density*, in the following sense. Similar to (7.27) and (7.28), a single component $P_{xy}(m)$ gives the cross-power in range $(m \pm 0.5)/N$ Hz-s. And as illustrated in Figure 7.11, we add the components at positive and negative frequencies, that is, $P_{xy}(m)$ and $P_{xy}(-m) = P_{xy}(N-m)$, to get the total power in this frequency range. But in this case, these components are complex conjugates in accordance with (3.11). The development that led from (7.20) to (7.22) in this case leads to the following:

$$\begin{aligned}\text{Average cross-power} &= \frac{1}{N}\sum_{n=0}^{N-1} x_n y_n = \frac{1}{NT}\int_0^{NT} x(t)y(t)dt \\ &= \varphi_{xy}(0) = \frac{1}{N^2}\sum_{m=0}^{N-1} X'_m Y_m = \frac{1}{N}\sum_{m=0}^{N-1} P_{xy}(m)\end{aligned} \tag{7.37}$$

The sum on the right is real, because $P_{xy}(0)$ and $P_{xy}(N/2)$ are real, and $P_{xy}(m)$ and $P_{xy}(N-m)$ are conjugates for the other values of m, and also, of course, because $\varphi_{xy}(0)$ is real.

Thus, only the real part of the cross-periodogram contributes to the cross-power. In physical terms, if $v(t)$ and $i(t)$ are voltage and current waveforms, the average cross-power given by (7.37) is the actual power in watts. For example, if $v(t)$ and $i(t)$ are both unit sine waves, the average power is the average product, or 0.5 watt; if $v(t)$ is a sine wave and $i(t)$ is a cosine wave, the average power is again the average product, or zero.

Another important use of the cross-power spectrum comes from the following relationship, which may be derived just as (7.34) was derived. The cross-periodogram of vectors x and y with periodic extension is the DFT of the correlation function $\varphi_{xy}(k)$; that is,

$$\text{DFT}\{\varphi_{xy}(k)\} = P_{xy}(m); \quad m = 0, 1, \ldots, N-1 \tag{7.38}$$

Thus, $P_{xy}(m)$ tells us how the cross-correlation function is distributed over frequency. In this sense, the magnitude of $P_{xy}(m)$, that is, $|P_{xy}(m)|$, is a measure of the *coherence* of signals x and y at different frequencies. A function called the *Magnitude-Squared Coherence* function (MSC)[19,22,23] is defined as

$$\text{Magnitude-squared coherence:}\quad \Gamma^2_{xy}(\nu) = \frac{|P_{xy}(\nu)|^2}{P_{xx}(\nu)P_{yy}(\nu)} \tag{7.39}$$

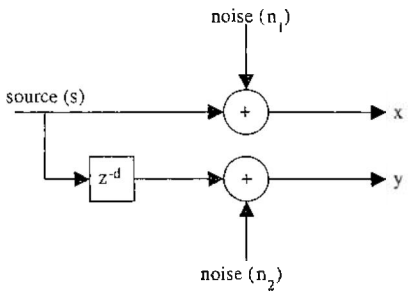

FIGURE 7.21
Generation of signals x and y, which were used to produce the power spectra in Figure 7.22.

The MSC is a normalized measure of the coherence of signals x and y at ν Hz-s in the sense that it indicates whether the cross-correlation function, $\varphi_{xy}(k)$, has a component at that frequency.

The MSC is useful in situations where the source of a signal is weak and inaccessible, and only noisy versions of the signal can be recorded by placing sensors away from the source. Medical situations where invasive procedures are ruled out, or geophysical applications where the source cannot be reached, are examples. Then, one may use multiple sensors and look for coherence between pairs of sensor outputs using the MSC. (The concept also extends to more than two sensors using multidimensional spectra.)

The MSC concept is illustrated in the situation shown in Figure 7.21, where two signals, x and y, are recorded by placing sensors where signals from an unknown source (s) are able to be received. In this example, white Gaussian noise (n_1) is added to s to produce x, and *independent* white Gaussian noise (n_2) is added to a delayed version of s to produce y. The power spectrum of s, $P_{ss}(\nu)$, is shown on the left in Figure 7.22, with $P_{xx}(\nu)$ and $P_{yy}(\nu)$ in the center of Figure 7.22. In situations like this, the signal quality is expressed in terms of the *signal-to-noise ratio* (SNR), which is

$$\text{SNR} = \frac{\text{signal power}}{\text{noise power}} \tag{7.40}$$

In this example, s is limited to the frequency range [0.1,0.3] Hz-s, and in this range, the SNR is 0.03, which is representative of a very noisy signal.

The magnitude of the cross-spectrum, $|P_{xy}(\nu)|$, is shown on the right in Figure 7.22. All four spectra are based on a record length of 10^5 samples and periodograms of length 20 with 50% segment overlap. The segment window in each case was a tapered boxcar. The cross-spectrum magnitude, which is the nonnormalized version of the MSC in (7.39), illustrates the "recovery" of the signal spectrum, $P_{ss}(\nu)$, in this situation. Note that all four spectra are

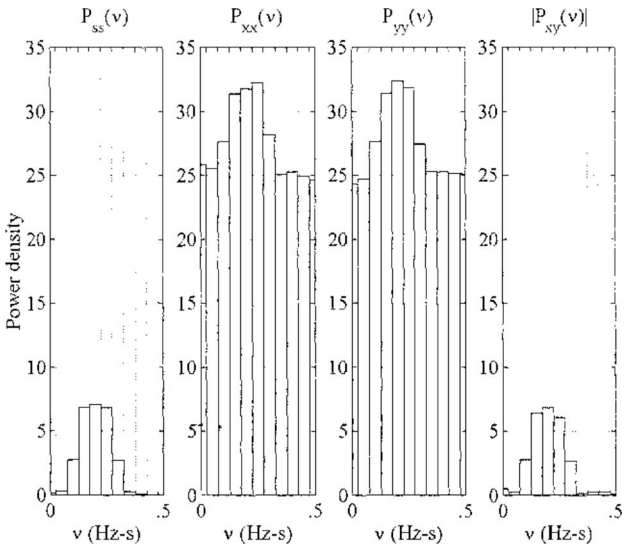

FIGURE 7.22
Power density spectra for the signals in Figure 7.21. Each spectrum is based on a recording of 10^5 samples of the signal(s). The SNR in this case was 0.03.

plotted on the same scale, and $|P_{xy}(\nu)|$ is (theoretically) the same as $P_{ss}(\nu)$, because n_1 and n_2 are statistically independent and, therefore, uncorrelated. Other illustrations of MSC may be found at the end of this chapter in Exercises 14 and 15.

Thus, we have seen that the cross-power spectrum has meaning in terms of electrical power (and similarly in terms of mechanical power), and that it is also useful in various ways as a measure of the distribution over frequency of the cross-correlation of two signal vectors.

7.9 Algorithms

Besides the various MATLAB functions that form the basis for statistical signal analysis, three additional functions included with the text were used to help produce examples in this chapter. First is the function

$$[f, xmin, xmax] = freq(x) \tag{7.41}$$

This function computes the frequency function (f_n) given by (7.3), as well as the minimum and maximum elements, of any integer array or vector, x. (If the elements of x are not integers, they are rounded.)

The second function is

$$bar2(v,p,color) \tag{7.42}$$

This is a simple modification of the MATLAB function bar, which may be used to plot continuous amplitude distributions or power spectra.

The third function is

$$[P, nsgmts] = pds(x, y, N, windo, overlap) \tag{7.43}$$

which may be used to compute a power spectral estimate as an average of periodograms. The segment size is N. When $y = x$, $P(1:N)$ is $P_{xx}(v)$. When x and y are two different signal vectors, $P(1{:}N)$ is $P_{xy}(v)$. The second output (*nsgmts*), if desired, gives the number of segments the function was able to use, given N and the amount of overlap, the latter being specified within the range (0,1).

7.10 Exercises

General Instructions: (1) Whenever you make a plot in any of the exercises, be sure to label both axes so we can tell exactly what the units mean. (2) Make continuous plots unless otherwise specified. "Discrete plot" means to plot discrete symbols, unconnected unless otherwise specified. (3) In plots, and especially in subplots, use the *axis* function for best use of the plotting area. (4) In exercises that require random numbers, initialize the seed to "123."

1. Complete the following:
   a. Given $y(t) = ax(t) + b$, begin with the definition of the mean in (7.4) and prove the formula for μ_y in (7.8).
   b. Begin with the definition of variance in (7.5) and prove the formula for σ_y^2 in (7.8).
2. Let $y = \sin x$. Assuming x is uniformly distributed in the range $(0, 2\pi)$, use (7.7) over a range of x, where y increases monotonically, say $x = (0, \pi/2)$, to derive the probability density function of y. Make a plot of $p(y)$ vs. y over the range $y = (-1,1)$.
3. Generate 5000 samples of a continuous uniform random variate with zero mean and average power equal to 27. Make a single plot of the theoretical continuous probability density, and on the same graph make a histogram plot of the sample amplitude distribution, using nine bars of equal width that exactly span the range of the variate.

4. Generate 5000 samples of a continuous Gaussian random variate with mean equal to 6 and average power equal to 4. Make a single plot of the theoretical continuous probability density, and on the same graph, make a histogram plot of the sample amplitude distribution, using bars centered at integers ranging from $\mu_x - 3\sigma_x$ to $\mu_x + 3\sigma_x$. Comment on the integral of the histogram plot in this case.
5. Is the amplitude distribution of $x + y$ equal to $p(x) + p(y)$? Prove your answer.
6. Suppose $x(t)$ is a Gaussian signal with mean equal to zero and power equal to 10. At any time t, determine the following:
   a. What is the probability that $x(t)$ lies in the range [0,5]?
   b. What is the probability that $|x(t)|$ lies in the range [1,2]?
7. For the signal vector (x), use the 5000-sample sequence in Exercise 3. Make three subplots. On the left, make a *continuous* plot of the periodogram of x over the range [0,0.5] Hz-s. In the center, make a bar plot similar to Figure 7.11, but only over [0,0.5] Hz-s, using half-overlapping 100-sample segments. On the right, make a similar bar plot using 40-sample segments. Use [0,100] for the periodogram range on all three plots.
8. Generate 1200 samples of uniform white noise with zero mean and average power equal to 3.
   a. Plot the amplitude distribution as a histogram with each bar width equal to 0.5.
   b. Make two subplots, each with a power density spectrum like Figure 7.11. For the left-hand plot, use nonoverlapping segments of length 80. For the right-hand plot, use segments of length 80 that overlap 50%. Use the tapered boxcar window in both cases.
   c. How does the integral of each plot compare with the theoretical average power? How do the two spectra compare with each other and with the theoretical power spectrum?
9. Generate 5000 samples of

$$x_k = \sin(2\pi k/50) + \sqrt{2}\cos(2\pi k/20)$$

Test the operation of function *pds* by making two subplots. On the left, plot power density vs. Hz-s using segment size 100, a boxcar window, and no overlap. On the right, make a similar plot using the Hamming window and 50% overlap. Make the plots over the range [0,0.5] Hz-s as in Figure 7.17.

   a. Explain the differences between the two plots.
   b. Equate the average power in x to the integral of each plot.

10. Generate 8192 samples of uniform white noise with mean zero and average power equal to 3 in vector x. Filter x to produce vector y. Use an FIR lowpass filter with Hamming window, 31 weights, and cutoff at 12.5 Hz. The time step between samples is 0.02 s.
    a. Explain how you would compute the theoretical power density spectrum of y.
    b. Estimate the power density spectrum using 64-sample segments, 50% overlap, and the Hamming window. Make a bar plot of the spectrum over the range [0,0.5] Hz-s, and overlay a continuous plot of the theoretical spectrum.
11. Describe how a window is *normalized*. In a 2 × 2 array of subplots, plot nonnormalized and normalized versions of the tapered rectangular, Hamming, Tent, and Blackman windows. Begin on the upper left with normalized and nonnormalized plots of the tapered rectangular window.
12. Generate $N = 8000$ samples of uniform white noise with mean = 0 and average power = 3.
    a. Plot the amplitude distribution as a histogram with each bar width equal to 0.5.
    b. Make three subplots. On the left, make a continuous plot of the periodogram of all N samples vs. frequency from –0.5 to 0.5 Hz-s. In the middle, make a bar plot of the power density using nonoverlapping segments of length 80. On the right, make a similar plot of 75%-overlapping segments of length 80. Use the Hamming window.
13. Generate vector x having 8192 uniform white noise samples with average power equal to one. Generate y as a similar vector of Gaussian white noise. Make four subplots. On the upper left, plot the amplitude distribution of x using a histogram with 15 bars. On the lower left, plot the power density of x over [0,0.5] Hz-s using half-overlapping segments of length 32 and the Hamming window. Do the same on the right for y, using amplitude range $\pm 3\sigma$.
14. Let k = [0:3999]. Generate vector $x = \sin(0.1\pi k)$. Then, generate vector $y = \cos(0.1\pi k) + n$, where n is a Gaussian white noise vector with zero mean and average power 100.
    a. Assuming the "signal" is the cosine wave, what is the SNR of y in dB?
    b. Make two subplots. On the left, make a bar plot of the power density spectrum of y over [0,0.5] Hz-z. Use 75% overlapping segments of length 40 and the boxcar window. On the right, make a similar plot of $|P_{xy}(\nu)|$, the magnitude of the cross-power density of x and y. Observe the differences in the two plots.

15. Generate $N = 8000$ samples of signals x and y in Figure 7.21, using $s = \sin(0.1\pi[0 : N-1])$, $d = 2$, and independent white Gaussian noise with power 50 for n_1 and n_2. Now let x and y simulate signals from two sensors, in which the signal from the unknown source, s, may or may not be present.
    a. Plot x and y, one above the other, in two subplots. Is there any sign of the source in these plots?
    b. Make three subplots. Use segment length 20, the boxcar window, and 75% overlap. From left to right, plot, $P_{xx}(\nu)$, $P_{yy}(\nu)$, and the mean-squared coherence, $\Gamma^2_{xy}(\nu)$, vs. ν over range $[0,0.5]$. Is there any sign of the source in these plots? If so, explain.

References

1. Shammugan, K.S., *Random Signals: Detection, Estimation, and Data Analysis*, Wiley, New York, 1988, chap. 2.
2. Douglas, C.M. and Runger, G.C., *Applied Statistics and Probability for Engineers*, 2nd ed., John Wiley & Sons, New York, 1999.
3. Derman, C., Gleser, L.J., and Olkin, I., *A Guide to Probability Theory and Application*, Holt, Rinehart and Winston, New York, 1973.
4. Dwass, M., *Probability Theory and Applications*, W.A. Benjamin, New York, 1970.
5. Feller, W., *An Introduction to Probability Theory and Its Applications*, 2nd ed., vol. 1, Wiley, New York, 1957.
6. Blanc-LaPierre, A. and Fortet, R., *Theory of Random Functions*, vol. 1, Gordon and Breach, New York, 1965.
7. Jenkins, G.M. and Watts, D.G., *Spectral Analysis and Its Applications*, Holden-Day, San Francisco, 1968.
8. Bendat, J.S. and Piersol, A.G., *Random Data: Analysis and Measurement Procedure*, John Wiley & Sons, New York, 1971, chap. 9.
9. Otnes, R.K. and Enochson, L., *Digital Time Series Analysis*, John Wiley, New York, 1972.
10. Koopmans, L.H., *The Spectral Analysis of Time Series*, Academic Press, New York, 1974.
11. Kay, S., *Modern Spectral Estimation: Theory and Application*, Prentice Hall, Englewood Cliffs, NJ, 1987.
12. Marple, S.L., *Digital Spectral Analysis with Applications*, Prentice Hall, Englewood Cliffs, NJ, 1987.
13. Oppenheim, A.V. and Schafer, R.W., *Discrete-Time Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989, chaps. 8, 11.
14. Rabiner, L.R. and Gold, B., *Theory and Application of Digital Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1975, chap. 6.
15. Richards, P.I., Computing reliable power spectra, *IEEE Spectrum*, 4, 83, Jan. 1967.
16. Welch, P.D., The use of the fast Fourier transform for the estimation of power spectra, *IEEE Trans.*, AU-15, 70, June 1967.
17. Bingham, C., Godfrey, M.D., and Tukey, J.W., Modern techniques of power spectrum estimation, *IEEE Trans.*, AU-15, 56, June 1967.

18. Yuen, C.K., A comparison of five methods for computing the power spectrum of a random process using data segmentation, *Proc. IEEE*, 65, 984, June 1977.
19. Hinich, M.J. and Clay, C.S., The application of the discrete Fourier transform in the estimation of power spectra, coherence, and bispectra of geophysical data, *Reviews of Geophysics*, 6, 347, Aug. 1968.
20. Carter, G.C. and Nuttall, A.H., On the weighted overlapped segment averaging method for power spectral estimation, *Proc. IEEE*, 68, 1352, 1980.
21. Nuttall, A.H. and Carter, G.C., A generalized framework for power spectral estimation, *IEEE Trans.*, ASSP-28, 334, June 1980.
22. Carter, G.C., Knapp, C.H., and Nuttall, A.H., Estimation of the magnitude-squared coherence function via overlapped FFT processing, *IEEE Trans.*, AU-21, 337, Aug. 1973.
23. Benignus, V.A., Estimation of the coherence spectrum and its confidence interval using the fast Fourier transform, *IEEE Trans.*, AU-17, 145, June 1969.

8

Least-Squares System Design

8.1 Introduction

The least-squares principle is widely applicable to the design of digital signal processing systems. In Chapter 2, we saw how to fit a general linear combination of functions to a desired function or sequence of samples, and how the finite Fourier series, as an example, provides a least-squares fit to a sequence of data. In Chapter 5, Exercise 5, we saw that the FIR filter gain may be expressed as a Fourier series in frequency, and is therefore a least-squares approximation to the ideal rectangular gain function. In this chapter, we wish to extend the least-squares concept to the design of other kinds of signal processing systems. We will see how several kinds of DSP systems may be realized using a common linear least-squares approach.

First, we describe briefly a variety of tasks involving *prediction, modeling, equalization,* and *interference canceling,* where least-squares design is useful. We show that all these tasks lead to the same least-squares design problem, which is to match a particular signal to a desired signal so that the difference between the two signals is minimal in the least-squares sense.

Next, we discuss the solution to the least-squares system design problem, which, for nonrecursive systems, amounts to the inversion of a matrix of correlation coefficients, similar to the symmetric coefficient matrix in Chapter 2, (2.10). We show that finding this solution is equivalent to finding the minimum point on a quadratic mean-squared-error performance surface (a concept that will become useful when we discuss *adaptive signal processing* in Chapter 9).

We also include various examples of least-squares design. The examples illustrate a variety of system configurations used in different applications, and also different ways to estimate correlation coefficients. However, the examples do not even begin to cover all of the many applications of this topic. A number of additional applications are included in the exercises, and therefore, the exercises are an important part of this particular chapter. Furthermore, this chapter forms the basis for adaptive signal processing, which is the subject of Chapter 9.

8.2 Applications of Least-Squares Design

In this section, we describe system configurations where the least-squares design is applicable. The first configuration, illustrated in Figure 8.1, is the *linear predictor*. The prediction concept is illustrated in its simplest form in the upper diagram. In the least-squares design, the coefficients, or *weights*, of a causal linear system, $H(z)$, are adjusted to minimize the *mean-squared error*, $E[e_k^2]$, thereby making the system output, g_k, the least-squares approximation to a desired signal, d_k. In this case, d_k is the same as the input, s_k, and must be "predicted" using the past history of s_k. That is, d_k must be predicted in terms of s_k delayed by m samples and processed through $H(z)$. Again, the least-squares design process consists of adjusting the weights of $H(z)$ to make this processed version of the history of the input the best "prediction" of the present sample value, s_k.

The upper diagram in Figure 8.1 with a unit delay, that is, with $m = 1$, is the most common form of the linear predictor, even though it does not actually predict a future value of the input. In most signal processing applications, the error, e_k, rather than a future value of s_k is needed. If an actual prediction of a signal is needed, the augmented form in the lower diagram

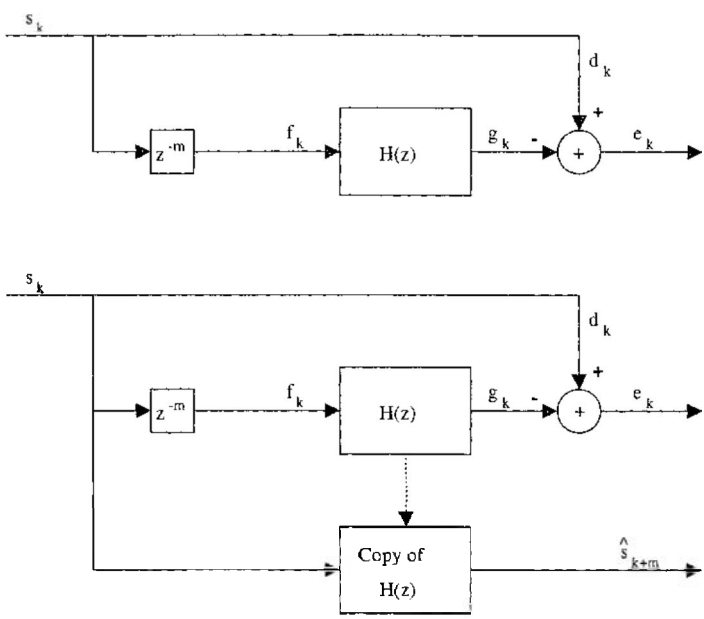

FIGURE 8.1
Least-squares linear prediction. In the upper diagram, g_k is the prediction of the current input, s_k, given the past history of s. In the lower diagram, $\hat{s}_k$ is the prediction of a future input, s_{k+m}, based on the history of s.

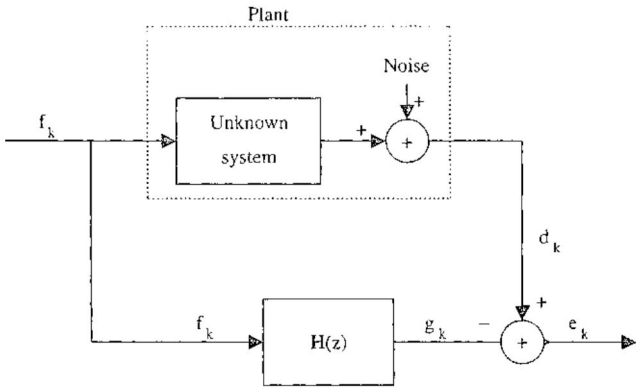

FIGURE 8.2
Least-squares modeling or system identification. When the noise is independent of the input, f, $H(z)$ becomes a "model" of the unknown system in the sense that its output, g_k, is a least-squares approximation to the output of the unknown system.

of Figure 8.1 may be implemented. Here, s_k is processed through a copy of the predictor, $H(z)$, which produces the predicted future value, $\hat{s}_{k+m}$.

The linear predictor is useful in waveform encoding and data compression,[12,19] spectral estimation,[9,10,15,16] spectral line enhancement,[14] event detection,[13] and other areas. Its effect in all these applications, as one might guess from the previous discussion, is to produce a signal e_k, which is a *decorrelated* or *whitened* version of s_k. The input is a signal that may be predictable in terms of its past values, and the output, one might say, is everything that is left after the predictable part is taken out.

The second configuration in which least-squares design is applicable is illustrated in Figure 8.2 and is called *modeling* or *system identification*. Here, a linear system, $H(z)$, models or identifies an unknown "plant" (see Section 8.8) consisting of an unknown system with internal noise. The least-squares design forces the linear system output, g_k, to be a least-squares approximation to the desired plant output, d_k, for a particular input signal, f_k. When f_k has spectral content at all frequencies and when the plant noise contributes at most a small part of the power in d_k, we expect $H(z)$ to be similar to the transfer function of the plant's unknown system. Note, however, that $H(z)$ is not necessarily a least-squares approximation, as it was, for example, in Chapter 5. Thus, the modeling concept is applicable where the best approximation to a signal, rather than to a transfer function, is the objective. The type of modeling illustrated in Figure 8.2 has a wide range of applications, including modeling in the biological, social, and economic sciences,[22] in adaptive control systems,[3,8] in digital filter design,[23] and in geophysics.[3]

Another application of least-squares system design, known as *inverse modeling* or *equalization*, is illustrated in Figure 8.3. Here, the desired output of $H(z)$, again labeled d_k, is a delayed version of the input signal. To cause the

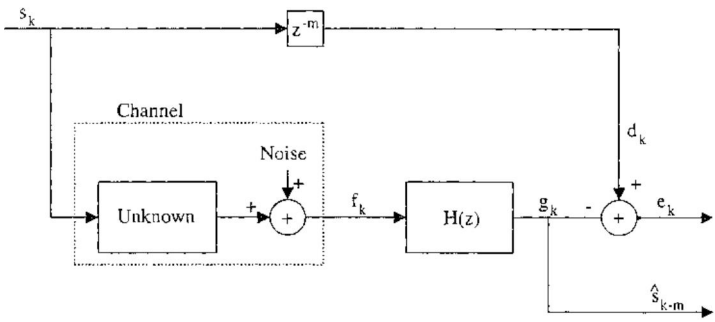

FIGURE 8.3
Least-squares inverse modeling, or equalization. When the noise is independent of the input, $H(z)$ becomes the inverse of the channel, and thus "equalizes" the effect of the channel on the input, producing a delayed least-squares approximation to the input.

output g_k to approximate d_k, $H(z)$ is adjusted to model the inverse of, or *equalize*, the unknown system with internal noise. When the unknown system and the linear system are both causal, the delay, z^{-m}, is used to compensate for the propagation delay through the two systems in cascade. When the internal noise of the unknown system is only a small part of f_k, the output of the equalizer, g_k, is a good approximation to the delayed input, s_{k-m}. Hence, the linear system is used here to invert, or equalize, the effect of the unknown system on the input signal.

Equalization is used in communication systems to remove distortion by compensating for nonuniform channel gain and multipath effects and to improve the signal-to-noise ratio if the channel introduces band-limited noise.[3,17,18] Equalization and inverse modeling are also used in adaptive control,[3] speech analysis,[7,15,16] deconvolution,[20] digital filter design,[3,23] and other areas.

Our final example of a configuration where least-squares design is used, *interference canceling*, is illustrated in Figure 8.4. The interference-canceling principle[1,3,21] is applicable in cases where there is a signal, s_k, with additive noise, n_k, and also a source of correlated noise, n'_k. Ideally, n_k and n'_k are correlated with each other but not with s_k, although the principle is applicable even if the signal and noise are correlated. The least-squares design objective is to adjust $H(z)$ so that its output, g_k, is a least-squares approximation to n_k, thereby canceling, by subtraction, the noise from the incoming waveform. When the signal and noise are independent, this is equivalent to minimizing the mean-squared value of the error, $E[e_k^2]$, because the independent noise cannot be made to cancel the signal, s_k. The delay is placed in the configuration to compensate for propagation through the causal linear system and allow the noise sequences, n_k and n'_k, to be aligned in time.

Interference canceling is an interesting and sometimes preferable alternative to bandpass filtering for improving the signal-to-noise ratio. For example, suppose we have an underground seismic sensor that receives, in addition to a seismic signal component, s_k, an acoustic noise component, n_k, coupled

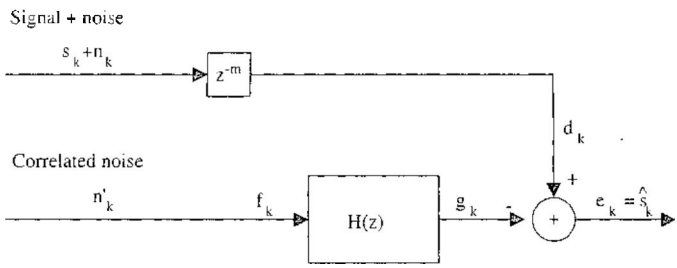

FIGURE 8.4
Least-squares noise cancellation. When the noise (n_k) is independent of the signal, and correlated noise (n'_k) can be measured independently, $H(z)$ produces a least-squares approximation to n_k, and thus, improves the SNR, even though s_k and n_k may have overlapping spectra.

into the ground from the atmosphere above, and the two components have similar spectra so that n_k cannot be removed from $(s_k + n_k)$ using bandpass filtering. We could then add an above-ground microphone to the system to receive n'_k, an acoustic component correlated with (but not exactly the same as) n_k, and process the signals as in Figure 8.4 to reduce the acoustic noise and increase the signal-to-noise ratio. The number of examples of this type is limited only by one's imagination.

Although the applications illustrated in Figures 8.1 through 8.4 are distinctly different, they all involve the same least-squares design problem. The important features may be summarized as follows. There is a linear system, $H(z)$, with adjustable coefficients. These coefficients are adjusted to cause the output, g_k, of the linear system to be a least-squares approximation to the desired signal, d_k, and thereby minimize the mean-squared error, $E[e_k^2]$. We now proceed to examine the nature of the resulting least-squares design problem, using a geometrical interpretation.

8.3 System Design via the Mean-Squared Error

If we compare Figures 8.1 through 8.4, we can see a common least-squares design problem, which is illustrated in Figure 8.5. The parameters of a causal linear system, $H(z)$, are to be adjusted or selected to minimize a mean-squared error, $E[e_k^2]$. [Actually, there is no need in what follows to assume that $H(z)$ is causal, but we assume causality for convenience, and to include real-time applications.] For example, if $H(z)$ is a linear system of the form $B(z)/A(z)$, the parameters to be selected are b and a, the coefficient vectors with transforms of $B(z)$ and $A(z)$. The error, e_k, is the difference between the desired signal, d_k, and the linear system output, g_k:

$$e_k = d_k - g_k \tag{8.1}$$

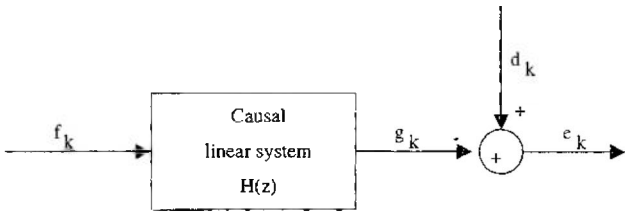

FIGURE 8.5
Essential ingredients common to Figures 8.1 through 8.4. The weights of a causal linear system, $H(z)$, are adjusted to minimize the MSE, that is, the mean value of e_k^2, and thereby make g_k a least-squares approximation to d_k.

Let us now assume that the signals in Figure 8.5 are stationary so that expected values and correlation functions are defined. Then the mean-squared error (MSE) is

$$\begin{aligned} \text{MSE} \equiv \mathrm{E}[e_k^2] &= \mathrm{E}[(d_k - g_k)^2] \\ &= \mathrm{E}[d_k^2 + g_k^2 - 2d_k g_k] \\ &= \varphi_{dd}(0) + \mathrm{E}[g_k^2] - 2\mathrm{E}[d_k g_k] \end{aligned} \tag{8.2}$$

The last result follows from the definition of the correlation function in Chapter 3, (3.2). For the causal linear system in Figure 8.5, we have

$$g_k = \sum_{n=0}^{N-1} h_n f_{k-n}; \quad 0 \le k < \infty \tag{8.3}$$

where vector h is the impulse response of $H(z)$. Using this in (8.2), we obtain

$$\begin{aligned} \text{MSE} &= \varphi_{dd}(0) + \mathrm{E}\left[\sum_{n=0}^{N-1}\sum_{m=0}^{N-1} h_n h_m f_{k-n} f_{k-m}\right] - 2\mathrm{E}\left[d_k \sum_{n=0}^{N-1} h_n f_{k-n}\right] \\ &= \varphi_{dd}(0) + \sum_{n=0}^{N-1}\sum_{m=0}^{N-1} h_n h_m \varphi_{ff}(n-m) - 2\sum_{n=0}^{N-1} h_n \varphi_{df}(-n) \end{aligned} \tag{8.4}$$

In the center term, f_{k-n} and f_{k-m} are displaced $n - m$ time steps from each other; hence, the expected product is $\varphi_{ff}(n - m)$. Similarly, the expected value of $d_k f_{k-n}$ is $\varphi_{df}(-n)$.

Our next objective is to formulate the minimization of (8.4) as a linear least-squares problem so we can apply the methods in Chapter 2 to its

solution for the optimal impulse response. Because the MSE in (8.4) is a quadratic function of the N samples in h, we can see that the gradient of the MSE with respect to these samples is a linear function of the samples; that is,

$$\frac{\partial \text{MSE}}{\partial h_k} = 2\sum_{n=0}^{N-1} \varphi_{ff}(n-k)h_n - 2\varphi_{fd}(k); \quad k = 0, 1, \ldots, N-1 \tag{8.5}$$

[To obtain this, we used the fact that $\varphi_{ff}(n)$ is an even function of n, and also substituted $\varphi_{fd}(n)$ in place of $\varphi_{df}(-n)$.] Setting this gradient vector equal to zero, we have the N equations we need to solve for the impulse response of the "optimal" $H(z)$ that minimizes the MSE in a stationary signal environment:

$$\begin{bmatrix} \varphi_{ff}(0) & \varphi_{ff}(1) & \varphi_{ff}(2) & \ldots & \varphi_{ff}(N-1) \\ \varphi_{ff}(1) & \varphi_{ff}(0) & \varphi_{ff}(1) & \ldots & \varphi_{ff}(N-2) \\ \varphi_{ff}(2) & \varphi_{ff}(1) & \varphi_{ff}(0) & \ldots & \varphi_{ff}(N-3) \\ \vdots & \vdots & \vdots & & \vdots \\ \varphi_{ff}(N-1) & \varphi_{ff}(N-2) & \varphi_{ff}(N-3) & \ldots & \varphi_{ff}(0) \end{bmatrix} \cdot \begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ \vdots \\ h_{N-1} \end{bmatrix} = \begin{bmatrix} \varphi_{fd}(0) \\ \varphi_{fd}(1) \\ \varphi_{fd}(2) \\ \vdots \\ \varphi_{fd}(N-1) \end{bmatrix} \tag{8.6}$$

Thus, we formulated the problem of minimizing the MSE as a linear least-squares problem. The coefficient matrix, called the *autocorrelation matrix* of the signal f, is known as a *Toeplitz* matrix on account of its special form. Notice that each column (or row) is a rotation of the *autocorrelation vector*, φ_{ff}, which is $N \times 1$. Let Φ_{ff} denote the coefficient matrix. In MATLAB, to create Φ_{ff} from the elements of φ_{ff}, we create an $N \times N$ *index array*. MATLAB indices must begin at one, so the elements of the index array are one plus the indices of φ_{ff} in (8.6), that is, 1 through N on the first row, 2 followed by 1 through $N-1$ in the second row, etc., and finally N, $N-1$,...,1 on the last row. The autocorrelation matrix, Φ_{ff}, is then created using the index array. For example, the following expressions, in which *index_mat* is the index matrix, will produce Φ_{ff} from the autocorrelation vector, *phi*:

```
N=length(phi);
q=[phi(N:-1:2);phi];
ix1=[0:N-1]'*ones[1,N];
ix2=ones(N,1)*[N:-1:1];
index_mat=ix1+ix2
Phi_ff=q(index_mat)
```

(8.7)

By running this algorithm with "phi" equal to a column vector of length $N = 3$ or 4 and allowing q and *index_mat* to be printed, one can observe how

the index matrix is used to create Φ_{ff} from the correlation vector, *phi*. For example, with *phi* equal [90, 80, 70]′ and therefore *q* equal [70, 80, 90, 80, 70]′,

```
index_mat =
     3  2  1
     4  3  2
     5  4  3
Phi_ff =
    90  80  70
    80  90  80
    70  80  90
```

(8.8)

The solution of (8.6) is the impulse response vector, *h*, of the optimal causal linear system that minimizes the MSE. If the system is nonrecursive, then $h = b$; that is, the elements of *h* are the weights of the optimal FIR filter. If the system is recursive, then *h* is the inverse transform of $B(z)/A(z)$, and the solution for *a* and *b* is, in general, a nonlinear problem. Therefore, we will restrict our discussion to the causal FIR configuration in Figure 8.6 with impulse response $h = b$.

We can write (8.4) and (8.6) using vector and array notation, again using the notation where $\varphi_{xy}(n)$ stands for the *n*th of *N* elements of column vector φ_{xy}, and assuming $b = h$ is also a column vector of length *N*. The two vector expressions are, respectively,

$$\begin{aligned} &(8.4): \quad \text{MSE} = \varphi_{dd}(0) + b'\Phi_{ff}b - 2b'\varphi_{fd} \\ &(8.6): \quad \Phi_{ff}b = \varphi_{fd} \end{aligned} \tag{8.9}$$

An algorithm due to Levinson[25,26] uses the special properties of Φ_{ff} to solve for *b* in (8.6). If Levinson's algorithm is not available, the methods of Chapter 2 may be used. The solution without the use of Levinson's algorithm and the resulting minimum MSE are given in Table 8.1, where we use b_{opt} to represent the solution to (8.6), that is, the optimal weight vector.

The result for the minimum MSE is proved by substituting the optimal weight expression into the MSE and noting that $\Phi'_{ff} = \Phi_{ff}$, $[\Phi_{ff}^{-1}\varphi_{fd}]' = \varphi'_{ff}\Phi_{ff}^{-1}$, and also $\varphi'_{fd}b_{\text{opt}} = b'_{\text{opt}}\varphi_{fd}$.

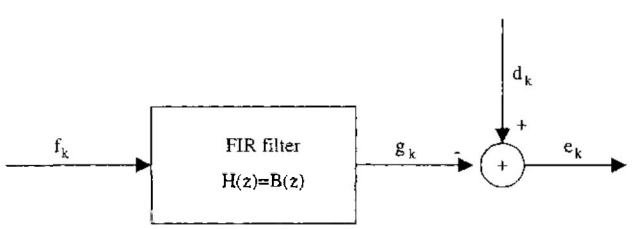

FIGURE 8.6
The same essential ingredients as in Figure 8.5, but now with $H(z)$ in the form of an FIR filter with weights $b = [b_0, b_1, \ldots, b_{N-1}]'$.

TABLE 8.1

Least-Squares Equations Using Correlation: Algebraic and MATLAB

Mean-squared error (MSE)	$\text{MSE} = \varphi_{dd}(0) + b'\Phi_{ff}b - 2b'\varphi_{fd}$ `MSE=phi_dd(1)+b'*Phi_ff*b-2*b'*phi_fd`
Optimal weights	$b_{\text{opt}} = \Phi_{ff}^{-1}\varphi_{fd}$ `bopt=Phi_ff\phi_fd`
Minimum MSE	$\text{MSE}_{\text{min}} = \varphi_{dd}(0) - \varphi'_{fd}b_{\text{opt}}$ `MSEmin=phi_dd(1)-phi_fd'*bopt`

We note in these equations that the MSE is, by its nature, always positive, and therefore, because it is a quadratic function of the weights, it describes a bowl-shaped surface in $(N + 1)$ dimensional Cartesian space. Thus, the MSE must have the single global minimum given by the optimal weight vector. This minimum might be distributed due to a nonsingular correlation matrix, that is, the bowl might have a flat bottom, but local minima cannot exist.

For some system design problems, the correlation vectors are known or assumed exactly. In other applications, they must be estimated. In still other applications, they may drift slowly with time. This leads to the design of *adaptive* signal-processing systems, which is the subject of the next chapter. Adaptive systems are systems that adjust the weight vector, b, continually as they continuously seek the solution to (8.6).

8.4 A Design Example

We now consider a simple example illustrating system design via the minimum MSE. Perhaps the easiest type of system to understand and to solve is the *least-squares predictor* with a simple periodic input, an example of which is shown in Figure 8.7. The predictor is an example of the upper diagram in Figure 8.1, with the delay (m) set to one time step. It is called a *one-step predictor*. For the signal s_k, we use a sine wave:

$$s_k = \sqrt{2}\sin(2\pi k/12); \quad -\infty < k < \infty \tag{8.10}$$

The correlation functions in (8.6) are found by averaging over one cycle (12 samples) of s_k. Using the fourth line of Table 1.2 in Chapter 1, we have

$$\begin{aligned} \varphi_{ff}(n) &= \frac{2}{12}\sum_{k=0}^{11}\sin\left(\frac{2\pi k}{12}\right)\sin\left(\frac{2\pi(k+n)}{12}\right) = \cos\left(\frac{2\pi n}{12}\right) \\ \varphi_{fd}(n) &= \frac{2}{12}\sum_{k=0}^{11}\sin\left(\frac{2\pi(k-m)}{12}\right)\sin\left(\frac{2\pi(k+n)}{12}\right) = \cos\left(\frac{2\pi(m+n)}{12}\right) \end{aligned} \tag{8.11}$$

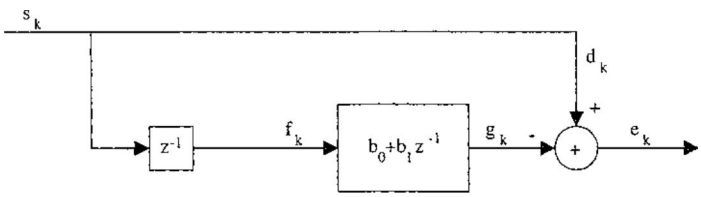

FIGURE 8.7
A simple example of the linear predictor in Figure 8.1, known as a "one-step predictor with two weights."

With the delay (m) set at one, these computations give us the following for the present example:

$$\Phi_{ff} = \begin{bmatrix} 1.0000 & 0.8660 \\ 0.8660 & 1.0000 \end{bmatrix}; \quad \varphi_{fd} = \begin{bmatrix} 0.8660 \\ 0.5000 \end{bmatrix} \tag{8.12}$$

We also note that, in this example, $\varphi_{dd} = \varphi_{ff}$, which is the first row or column of Φ_{ff}. Thus, the MSE in Table 8.1 for this example is

$$\text{MSE} = 1 + \begin{bmatrix} b_0 & b_1 \end{bmatrix} \cdot \begin{bmatrix} 1.0000 & 0.8660 \\ 0.8660 & 1.0000 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \end{bmatrix} - 2 \begin{bmatrix} b_0 & b_1 \end{bmatrix} \cdot \begin{bmatrix} 0.8660 \\ 0.5000 \end{bmatrix} \tag{8.13}$$

The MSE, a quadratic function of b, is plotted in Figure 8.8. The optimal weight values and the minimum MSE are, in accordance with Table 8.1,

$$b_{\text{opt}} = \Phi_{ff}^{-1} \varphi_{fd} = \begin{bmatrix} \sqrt{3} \\ -1 \end{bmatrix}; \quad \text{MSE}_{\text{min}} = \varphi_{dd}(0) - \varphi_{fd}' b = 0 \tag{8.14}$$

The surface in Figure 8.8 reaches zero at the point where the heavy lines cross, that is, where the weights are at their optimal values. Thus, the one-step predictor with optimal weights exactly cancels the signal, s_k, and the error, e_k, is zero.

The MSE is illustrated in Figure 8.8 as a three-dimensional bowl-shaped surface. A *contour plot* of the MSE is illustrated in Figure 8.9. Most of the time, the contour plot is preferable for analysis. It shows clearly how the MSE decreases to a minimum at the optimal value of b and gives a better picture of the gradient of the MSE surface. The four contours in Figure 8.9 were plotted with the following MATLAB instructions:

```
v=[.05  .15  .4  .8  1.8];
[c,h]=contour(b0,b1,mse,v,'k'); grid;
clabel(c);
```
(8.15)

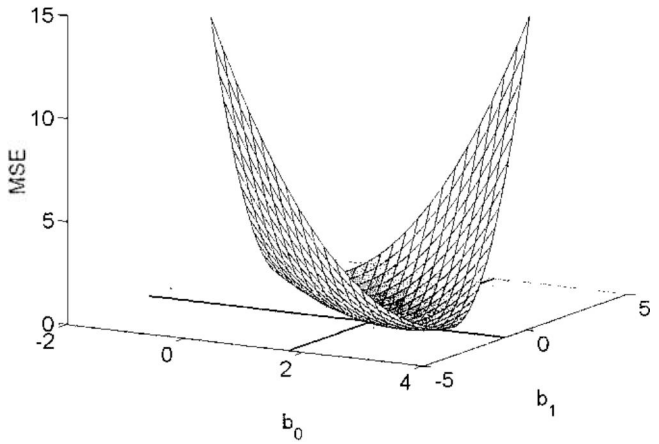

FIGURE 8.8
Mean-squared-error plot for the example in Figure 8.7 with $s_k = \sqrt{2}\sin(2\pi k/12)$.

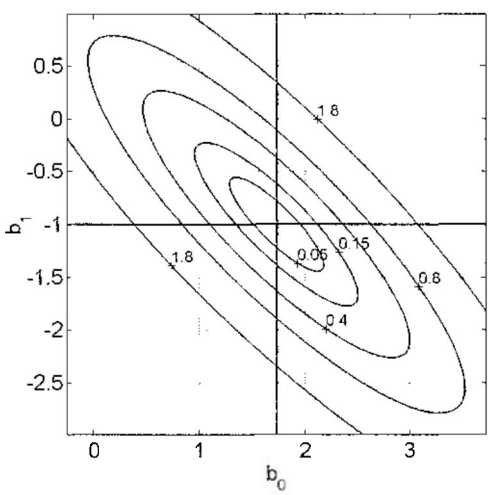

FIGURE 8.9
Contour plots, as if viewing a portion of the MSE in Figure 8.8 from directly above.

For the plot, the elements of vectors b_0 and b_1 were spaced evenly over the ranges shown, and *mse* was a matrix of MSE values computed (in this case) on a grid of 150^2 evenly spaced pairs of elements of b_0 and b_1. The *clabel* function was used to label the contours.

This example is artificial, because the signal is simple and is known exactly, and also because the filter has only two weights. It is meant to illustrate the

quadratic error surface in three dimensions and to show a simple derivation of the optimal weights.

As a final step in this simple example, let us observe the effect of adding a third weight, b_2, to the filter in Figure 8.7. With the third weight, the arrays in (8.12) are now

$$\Phi_{ff} = \begin{bmatrix} 1.0000 & 0.8660 & 0.5000 \\ 0.8660 & 1.0000 & 0.8660 \\ 0.5000 & 0.8660 & 1.0000 \end{bmatrix}; \quad \varphi_{fd} = \begin{bmatrix} 0.8660 \\ 0.5000 \\ 0.0000 \end{bmatrix} \tag{8.16}$$

In this case, Φ_{ff} is singular and has no inverse. That is, (8.16) imposes constraints on the weight vector b, but does not provide one specific vector. The constraints may be expressed as

$$\begin{aligned} b_0 - b_2 &= \sqrt{3} \\ b_0 + \sqrt{3}b_1 + b_2 &= 0 \end{aligned} \tag{8.17}$$

They imply a distributed minimum of the error surface, that is, a subspace in which the MSE is everywhere equal to zero. The solution for the optimal vector with $b_2 = 0$ is seen to agree with the two-weight solution in (8.14).

8.5 Least-Squares Design with Finite Signal Vectors

In the preceding two sections, we dealt with stationary signals described in terms of correlation functions, and derived least-squares design equations from this basis. These we summarized in Table 8.1. Now, we turn our attention to the design of FIR systems that are optimal with respect to finite signal vectors. There are many ways in which these systems may be applicable to the tasks of prediction, noise cancellation, etc., described in Section 8.2. Even in real-time applications, the high processing rates of DSP chips may be used in a *block processing* mode, in which the system is optimized for each block of time and processes the blocks of signals in order as they occur in real time.

When the signals in Figure 8.5 are finite vectors, each with K elements, the least-squares equations are quite similar to the equations in Table 8.1. Instead of the MSE, it is easier to use the *total squared error* (TSE) and avoid having to divide by K; that is, similar to (8.4),

$$\begin{aligned} \text{TSE} &= \sum_{k=0}^{K-1} d_k^2 + \sum_{k=0}^{K-1} \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} b_n b_m f_{k-n} f_{k-m} - 2 \sum_{k=0}^{K-1} \sum_{n=0}^{N-1} b_n d_k f_{k-n} \\ &= r_{dd}(0,0) + \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} b_n b_m r_{ff}(m,n) - 2 \sum_{n=0}^{N-1} b_n r_{df}(-n) \end{aligned} \tag{8.18}$$

Here, in place of the correlation functions of the stationary signals in (8.4), we substituted the *covariance functions* of the finite signal vectors, and these now completely determine the geometry of the least-squares design problem, just as the correlation functions did previously. The covariance functions in (8.18) are as follows:

$$\text{Autocovariance: } r_{xx}(m,n) = \sum_{k=0}^{K-1} x_{k-m}x_{k-n} = r_{xx}(n,m); \quad 0 \le (m,n) \le N-1$$

$$\text{Cross-covariance: } r_{xy}(n) = \sum_{k=0}^{K-1} x_k y_{k+n} = r_{yx}(-n); \quad 0 \le n \le N-1 \tag{8.19}$$

We note that covariance is defined in different ways. These definitions are convenient here, because they apply directly to the TSE formula (8.18).

Notice that the covariance functions, as defined in (8.19), require knowledge of the signal sequences beyond the range $0 \le k \le K-1$. Specifically, from the way the functions in (8.19) are used in (8.18), the following sequences are required:

$$\begin{aligned} f &= [f_{-N+1} \;\; f_{-N+2} \cdots f_0 \;\; \cdots \;\; f_{K-1}]' \\ d &= [d_0 \; d_1 \;\; \cdots \;\; d_{K-1}]' \end{aligned} \tag{8.20}$$

That is, these sequences are required to produce the error vector, e, on which the TSE is based. Depending on the application, as will be seen, elements f_{-K+1} through f_{-1} may be set to zeros, or they may be set equal to past values of f.

With the TSE expression (8.18), the principal results for the error surface and the least-squares weights in Table 8.1 are applicable, provided that the covariance functions in (8.19) are substituted for the correlation functions used previously. Table 8.2 is the covariance-equivalent of Table 8.1, with b_{opt} again representing the optimal weight vector.

In Table 8.2, as in Table 8.1, notice that the algebraic indices begin at zero and the MATLAB indices at one, and also that r_{df} in (8.18) is replaced with r_{fd} here in order to make the range of the index (n) positive. Also, the *autocovariance matrix*, R_{ff}, replaces the autocorrelation matrix in Table 8.1. Notice that

TABLE 8.2

Least-Squares Equations Using Covariance: Algebraic and MATLAB

Total-squared error (TSE)	$\text{TSE} = r_{dd}(0,0) + b'R_{ff}b - 2b'r_{fd}$
	`TSE=d'*d + b'*R_ff*b - 2*b'*r_fd`
Optimal weights	$b_{\text{opt}} = R_{ff}^{-1}r_{fd}$
	`bopt=R_ff\r_fd`
Minimum TSE	$\text{TSE}_{\text{min}} = r_{dd}(0,0) - r'_{fd}b_{\text{opt}}$
	`TSEmin=d'*d - r_fd'*bopt`

the autocovariance matrix is symmetric, that is, $r_{ff}(m, n) = r_{ff}(n, m)$, but is not Toeplitz, that is, the rows are, generally, not permutations of each other.

In summary, least-squares design using covariance is almost the same as least-squares design using correlation, the only essential difference being in the way we extend the ends of the signal vectors. Before considering more applications of these methods, we describe methods for computing the correlation and covariance functions for given signal vectors.

8.6 Correlation and Covariance Computation

In this section, we consider two simple methods for estimating correlation or computing covariance that are useful in a variety of applications but do not cover all cases. Here, we assume the functions are estimated from samples of the signals; however, especially in the case of correlation, this may not be true. The correlation functions may be derived theoretically from assumed properties of the signals, or they may be computed using the inverse DFT of the power spectrum as described in Chapter 7.

Suppose the cross-correlation function $\varphi_{xy}(n)$ is to be estimated in terms of two signal vectors, x and y. Suppose we allow these vectors to be extended periodically. Then, we may compute $\varphi_{xy}(n)$ as follows:

$$\varphi_{xy}(n) = \frac{1}{K}\sum_{k=0}^{K-1} x_k y_{k+n(\pm K)} = \varphi_{yx}(-n); \quad 0 \le |n| \le K-1 \tag{8.21}$$

The computation of $\varphi_{xy}(n)$ in this form requires K^2 products, and would be lengthy if (1) all K products were required and (2) K were large. Let us assume that both (1) and (2) are true and examine the computation in terms of a DFT product. From (8.21), we may conclude:

$$\varphi_{xy}(n) = \varphi_{yx}(-n) = \frac{1}{K}\sum_{k=0}^{K-1} y_k x_{k-n(\pm K)} = \frac{1}{K}\sum_{k=0}^{K-1} y_k x^r_{n-k(\pm K)}; \quad 0 \le |n| \le K-1 \tag{8.22}$$

where r is the operation in (6.4) that reverses the order of the elements of a vector. This result gives $\varphi_{xy}(n)$ in the form of a convolution, and thus, as the inverse DFT of a DFT product in accordance with (4.67). Furthermore, using $z = e^{j\omega T}$ in (6.5), we can see that the DFT of a reversed vector x^r is X', that is, the conjugate of the DFT of x. Therefore, when (4.67) is applied to (8.22), the result is

$$\boxed{\text{Correlation vector } \varphi_{xy} = \frac{1}{K}\text{DFT}^{-1}\{X'Y\}} \tag{8.23}$$

Although this result, which is analogous to (4.67), but for correlation instead of convolution, is of general interest, it is only useful if the *entire* autocorrelation vector is required. If K is the length of x and y, then a computation of N elements of φ_{xy} requires NK products. In Chapter 4, (4.69), we argued that the computation of (8.23) requires $12K \log_2(2K)$ real products. Therefore, the computation of $\varphi_{xy}(1:N)$ becomes more efficient using (8.23) only when

$$NK > 12K \log_2(2K), \quad \text{or} \quad N > 12\log_2(2K) \tag{8.24}$$

In least-squares applications, this condition is not very likely when K is large. In typical applications, K is at least an order of magnitude greater than N. Therefore, we usually prefer to compute correlation and covariance computations in the form of (8.21) rather than (8.23).

Three correlation functions are coded in the form of (8.21). The first is *autocorr*, which is illustrated in (8.25). The arguments are x, the signal vector, *type*, which is set to zero to extend x with zeros and one to extend x periodically, and N, the number of values of φ_{xx} to compute.

Autocorrelation example:
`x=[x1 x2 x3 x4]; phi=autocorr(x,type,3)`

	type=0	type=1
phi(1)	$(x_1x_1+x_2x_2+x_3x_3+x_4x_4)/4$	$(x_1x_1+x_2x_2+x_3x_3+x_4x_4)/4$
phi(2)	$(x_1x_2+x_2x_3+x_3x_4)/4$	$(x_1x_2+x_2x_3+x_3x_4+x_4x_1)/4$
phi(3)	$(x_1x_3+x_2x_4)/4$	$(x_1x_3+x_2x_4+x_3x_1+x_4x_2)/4$

(8.25)

The second correlation function is *crosscorr*, which has similar arguments and is illustrated in (8.26).

Cross-correlation example:
`x=[x1 x2 x3 x4]; y=[y1 y2 y3 y4];`
`phi=crosscorr(x,y,type,3)`

	type=0	type=1
phi(1)	$(x_1y_1+x_2y_2+x_3y_3+x_4y_4)/4$	$(x_1y_1+x_2y_2+x_3y_3+x_4y_4)/4$
phi(2)	$(x_1y_2+x_2y_3+x_3y_4)/4$	$(x_1y_2+x_2y_3+x_3y_4+x_4y_1)/4$
phi(3)	$(x_1y_3+x_2y_4)/4$	$(x_1y_3+x_2y_4+x_3y_1+x_4y_2)/4$

(8.26)

The third correlation function is *Phi* = *autocorr_mat*(x, *type*, N), which computes the N-element autocorrelation vector φ_{xx} using *autocorr* and then produces *Phi*, the $N \times N$ autocorrelation matrix Φ_{xx} using code similar to (8.7).

In the case of covariance as it is used in least-squares design, there is not much reason to compute an autocovariance vector. Therefore, there are just

two MATLAB functions, the first being $R = autocovar\_mat(x,N)$, which implements the computation of the elements $r_{xx}(m,n)$ in (8.19). For covariance computation, the signals (x in this case) are always extended with zeros. If nonzero values of the startup elements in (8.20) are necessary in the computation, then all signal vectors may be extended to the *left* with $N - 1$ elements, and K increased to $K + N - 1$. That is, in (8.20), we would append $N - 1$ startup values to the *beginning* of f, and $N - 1$ zeros to the beginning of d. With this understanding, the autocovariance equation in (8.19) becomes

$$r_{xx}(m,n) = \sum_{k=0}^{K-1} x_{k-m}x_{k-n} = \sum_{i=0}^{K-m-1} x_i x_{i+(m-n)}; \quad m \geq n \tag{8.27}$$

Comparing (8.21) with the second form of $r_{xx}(m,n)$, we note that as the signal vector becomes long enough to neglect end effects, the covariance function approaches K times the autocorrelation function. [The same is true for the cross-covariance function in (8.19).]

Because R is symmetric, we only need to compute elements for $m \geq n$. An example of the covariance computation with $N = 2$ is given in (8.28). As in the previous examples, the indices begin at one instead of zero.

Autocovariance example: `x=[x1 x2 x3 x4]; R=autocovar_mat(x,N)`	
`r(1,1)`	$x_1x_1+x_2x_2+x_3x_3+x_4x_4$
`r(2,1)`	$x_1x_2+x_2x_3+x_3x_4$
`r(2,2)`	$x_1x_1+x_2x_2+x_3x_3$

(8.28)

The second cross-covariance function is $r = crosscovar(x, y, N)$. The computation is made as in the cross-covariance formula in (8.19). An example is shown in (8.29).

Cross-covariance example: `x=[x1 x2 x3 x4]; y=[y1 y2 y3 y4]; r=crosscovar(x,y,3)`	
`r(1)`	$x_1y_1+x_2y_2+x_3y_3+x_4y_4$
`r(2)`	$x_1y_2+x_2y_3+x_3y_4$
`r(3)`	$x_1y_3+x_2y_4$

(8.29)

With the functions described in this section, and with the power of MATLAB in expressions such as those in Tables 8.1 and 8.2, it is easy to design least-squares systems for the kinds of applications described in Section 8.2. Examples are described in the following sections.

8.7 Channel Equalization

In this section, we consider another example of nonrecursive least-squares design in order to illustrate the use of the functions just described, and also to design a system different from the predictor designed previously. In this example, illustrated in Figure 8.10, we are to design a least-squares equalizer for an "unknown channel," which, in this case, is represented by an all-pole transfer function given by

$$U(z) = \frac{e^{-0.1}\sin(0.1\pi)z^{-1}}{1 - 2e^{-0.1}\cos(0.1\pi)z^{-1} + e^{-0.2}z^{-2}} \tag{8.30}$$

Unlike Figure 8.3, which is more realistic, we are not including noise in Figure 8.10. This description of the unknown channel assures us a simple design of the all-zero equalizer, $B(z)$, with zeros that must cancel the poles of $U(z)$. To obtain the equalizer, we assume $U(z)$ is unknown, but that we have sequences of the signals s_k and f_k, the latter being response of $U(z)$ to the input signal, s_k. These are shown in Figure 8.11, where s_k is seen to be an impulse at $k = 0$, and f_k is the impulse response, that is, the inverse transform of $U(z)$, which is a decaying sinusoid. (See Table 4.2, lines D and 4.) In MATLAB, we would express the signal vectors as follows:

```
s=[1, zeros(1,K-1)];
f=exp(-.1*k).*sin(.1*k*pi);
d=[zeros(1,m), s(1:K-m)];
```
(8.31)

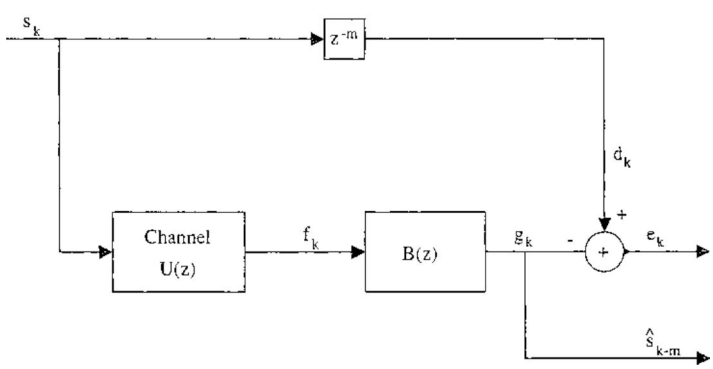

FIGURE 8.10
Equalizer design example. The "unknown" channel is specified by $U(z)$ in (8.30). The problem is to adjust the weights of $B(z)$ so that the squared difference between $\hat{s}_{k-m}$ and s_k is minimal.

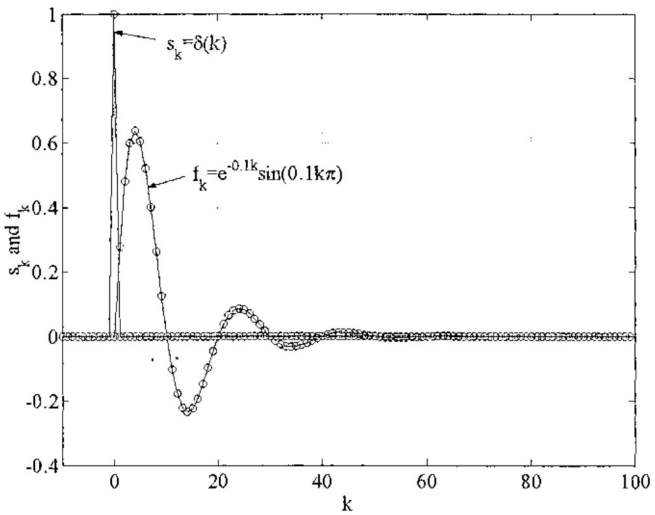

FIGURE 8.11
Signals in the equalizer example of Figure 8.10. The input signal, s_k, is a unit impulse, and f_k is the response of the unknown channel to s_k.

Notice that d is created by appending m zeros to the beginning of s, thus effecting the delay of m samples in Figure 8.10.

Thus, we are designing an equalizer that corrects the impulse response of the unknown channel so that its output, $\hat{s}_{k-m}$, is a delayed least-squares approximation to the signal s_k. Using the functions described in the preceding section, the optimal weights and minimum TSE for given values of N and m are computed as follows (see Table 8.2):

```
Rff=autocovar_mat(f,N);
rfd=crosscovar(f,d,N);
b=Rff\rfd;
TSEmin(N+1)=d*d'+b'*Rff*b-2*b'*rfd;
```
(8.32)

It is interesting to consider the effect of the delay (m) on the required number of weights, which is illustrated in Figure 8.12. The latter is a plot of the minimum TSE vs. the number of FIR weights for different values of the delay, m. It shows first that if the delay (m) is at least one, an equalizer that exactly compensates for the channel and drives the TSE to zero is realizable. Second, the plot shows that there is also an optimal delay in systems like this, in the sense that the number of equalizer weights is minimized.

8.8 System Identification

The next example in Figure 8.13 is one in which we are trying to identify (model) an unknown system or "plant". In this case, the plant has poles, so we cannot exactly match the desired signal with an FIR filter, that is, we cannot drive the TSE to zero using $B(z)$ to model the plant. In this example, as in the previous example, the transfer functions $U(z)$ and $B(z)$ in Figure 8.13 are

$$\begin{aligned} U(z) &= \frac{e^{-0.1}\sin(0.1\pi)z^{-1}}{1-2e^{-0.1}\cos(0.1\pi)z^{-1}+e^{-0.2}z^{-2}} \\ B(z) &= \sum_{n=0}^{N-1} b_n z^{-n} \end{aligned} \tag{8.33}$$

The input signal, f_k in Figure 8.13, is a random white signal with unit power. The input sequence is given by

```
f=randn(1,K)
```
(8.34)

That is, f is a white Gaussian noise sequence with K elements and unit power.

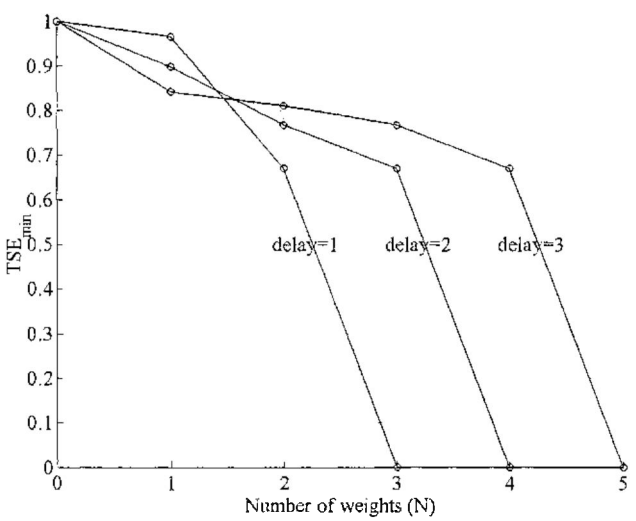

FIGURE 8.12
Minimum total squared error (TSE_{min}) versus number of optimal weights (N) in the equalizer example of Figure 8.10 for delay values 1, 2, and 3, illustrating the effect of the delay value.

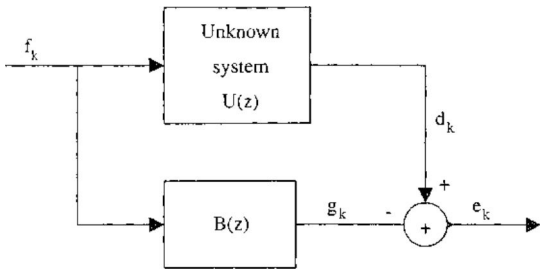

FIGURE 8.13
System identification example, in which $B(z)$ becomes a model of the unknown system when the total squared error is minimized.

As noted in Section 8.6, the covariance elements in the matrix R should approach scaled values of the correlation coefficients, $\varphi_{ff}(n)$, as K increases. Because f_k is a white random sequence with independent samples and unit power, we have

$$\varphi_{ff}(n) = \begin{cases} 0; & n \neq 0 \\ 1; & n = 0 \end{cases} \tag{8.35}$$

Thus, the theoretical autocorrelation matrix is diagonal, that is, $\Phi_{ff} = I$ theoretically, and from Table 8.1, the optimal weights are given by the cross-correlation vector; that is,

$$b = \varphi_{fd} \tag{8.36}$$

Now let us multiply the transfer-function relationship for $U(z)$ in Figure 8.13 by the conjugate of the transform of f to obtain

$$F'(z)D(z) = F'(z)F(z)U(z) \tag{8.37}$$

Using $\Phi_{fd}(z)$ and $\Phi_{ff}(z)$ to represent the z-transforms of the corresponding correlation functions, we can apply (8.23) to (8.37) and, noting that φ_{ff} is a unit impulse, and therefore, $\Phi_{ff}(z) = 1$, obtain

$$\Phi_{fd}(z) = \Phi_{ff}(z)U(z) = U(z) \tag{8.38}$$

Therefore, φ_{fd}, and thus the theoretical value of b, is the impulse response of $U(z)$, which, as we saw in (8.31), is

$$b_{n(\text{theoretical})} = e^{-0.1n}\sin(0.1n\pi) \tag{8.39}$$

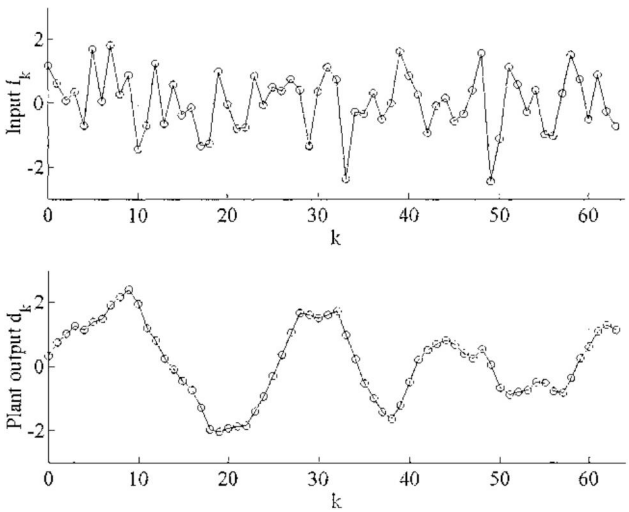

FIGURE 8.14
Signal vectors f and d in the system identification example.

Thus, with a broadband input signal, the least-squares weight vector b tends to match a finite portion of the impulse response of the unknown system. One might suppose that a very large length (K) of the random input sequence would be required for the covariance solution to match the correlation solution, but in fact, a short sequence suffices. As an example, the sequences of length $K = 64$ illustrated in Figure 8.14 were used to derive a least-squares model of size $N = 50$. These were produced with the following expressions:

```
K=64;
randn('seed',0);
f=randn(1,K);
c=exp(-.1);
d=filter[oc*sin(.1*pi)],[1 -2*c*cos(.1*pi) c^2], f);
```
(8.40)

The least-squares weight vector, b, is computed using the first three expressions in (8.32), with $N = 50$, and compared in Figure 8.15 with samples of the continuous plant impulse response, that is, the theoretical weights in (8.39). The two plots are nearly identical, and the minimum TSE in this example is on the order of 10^{-5}, indicating a nearly perfect model. Notice that the FIR filter length $N = 50$ covers essentially all of the nonzero portion of the impulse response of the unknown system.

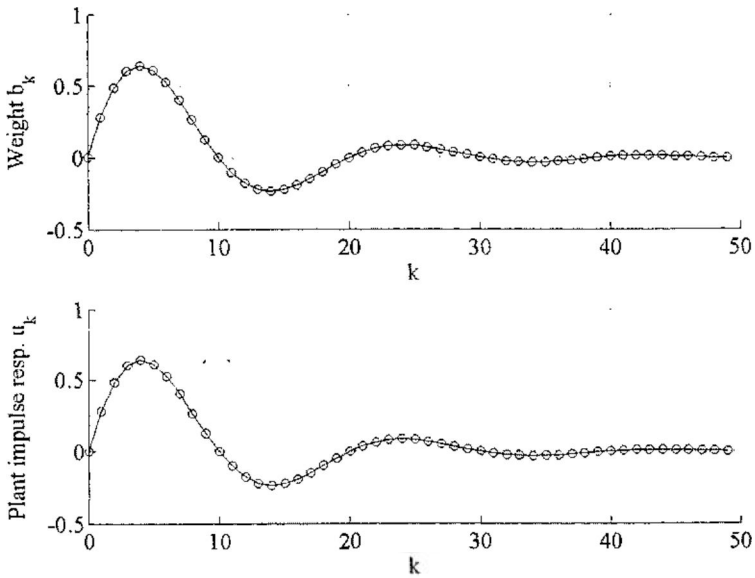

FIGURE 8.15
Impulse responses of the model, $B(z)$, and the unknown system. In this noiseless example, the model is nearly perfect in the sense that the TSE is nearly zero.

8.9 Interference Canceling

Our next example illustrates the interference-canceling concept in Figure 8.4 in the communication situation shown in Figure 8.16. A caller is speaking in a construction environment, where the acoustic noise is so loud that, when it is picked up by his telephone and added to his speech, the combined signal becomes unintelligible to the listener. Luckily, a DSP engineer is on hand to design the least-squares noise canceler, which operates by processing correlated noise picked up on a microphone in the same environment. The noise canceler works by *subtracting* the processed correlated noise, g_k, from the combined signal, $s_k + n_k$.

The reason for the necessity to subtract the noise instead of using a noise-canceling filter is shown in Figure 8.17. A 1-second sample of the speech signal, $s(t)$, is shown at the left along with the power density spectrum of the entire signal. The construction noise, $n(t)$, and its spectrum are on the right. The sampling rate in both cases is 40 KHz. We can see that filtering the combined signal, $s(t) + n(t)$, to improve the signal-to-noise ratio (SNR) might be possible to some degree but would also result in the loss of some of the signal. If $s(t)$ and $n(t)$ are independent, the configuration in Figure 8.16 should be a better choice for improving the SNR.